In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.impute import KNNImputer
```

In [109]:
```
df = pd.read_csv("C:/Users/THUSHAN/Desktop/competition/fraud_detection_dataset.csv")
df.head()
```

Out[109]:

	account_open_date	age	location	occupation	income_level	fico_score	delinquency_status	charge_off_status	number_of_credit_applications	debt_to_i
0	11/9/2023	56	Los Angeles	Teacher	40099	424.0	108	True	10	
1	9/11/2022	69	New York	Engineer	2050	483.0	0	False	1	
2	7/12/2020	46	Miami	Engineer	71936	566.0	0	False	1	
3	8/13/2024	32	Houston	Banker	15833	NaN	97	True	5	
4	7/27/2024	60	Houston	Lawyer	8574	787.0	0	False	1	

5 rows × 23 columns

In [110]:
```
df.shape
```

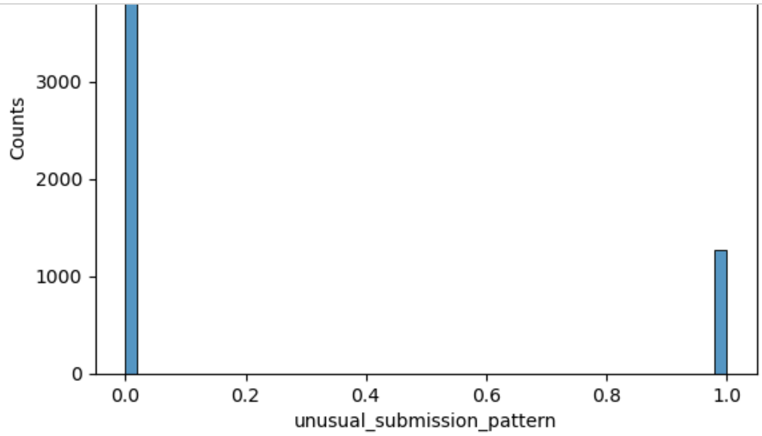
Out[110]: (7000, 23)

In [111]:
```
df.isnull().sum()
```

Out[111]:
```
account_open_date                          0
age                                        0
location                                   0
occupation                                 0
income_level                               0
fico_score                               210
delinquency_status                         0
charge_off_status                          0
number_of_credit_applications              0
debt_to_income_ratio                       0
payment_methods_high_risk                  0
max_balance                                0
avg_balance_last_12months                350
number_of_delinquent_accounts            700
number_of_defaulted_accounts               0
earliest_credit_account                    0
recent_trade_activity                      0
new_accounts_opened_last_12months          0
multiple_applications_short_time_period    0
unusual_submission_pattern               910
applications_submitted_during_odd_hours    0
watchlist_blacklist_flag                   0
public_records_flag                        0
dtype: int64
```

In [112]:
```
missing_values_columns = ['fico_score', 'avg_balance_last_12months','number_of_delinquent_accounts','unusual_submission_patt
```

In [113]:
```
for i in range(len(missing_values_columns)):
    sns.histplot(df[missing_values_columns[i] ], bins=50)
    plt.title(f"Distribution of {missing_values_columns[i] } Data ")
    plt.xlabel(missing_values_columns[i])
    plt.ylabel('Counts')
    plt.show()
```



In [114]:
```
df['fico_score'].fillna(df['fico_score'].median(), inplace=True)
```

In [115]:
```
imputer = KNNImputer(n_neighbors=2)

# Apply KNN Imputation on the selected columns

df_imputed1= imputer.fit_transform(df[['number_of_delinquent_accounts']])

df_imputed2=imputer.fit_transform(df[['unusual_submission_pattern']])
df_imputed3=imputer.fit_transform(df[['avg_balance_last_12months']])

# Replace the original value into column with imputed values

df['number_of_delinquent_accounts'] = df_imputed1[:, 0].round()
df['unusual_submission_pattern'] = df_imputed2[:, 0].round()
df['avg_balance_last_12months'] = df_imputed3[:, 0]
```

In [116]:
```
df.isnull().sum()
```

Out[116]:
```
account_open_date                        0
age                                      0
location                                 0
occupation                               0
income_level                             0
fico_score                               0
delinquency_status                       0
charge_off_status                        0
number_of_credit_applications            0
debt_to_income_ratio                     0
payment_methods_high_risk                0
max_balance                              0
avg_balance_last_12months                0
number_of_delinquent_accounts            0
number_of_defaulted_accounts             0
earliest_credit_account                  0
recent_trade_activity                    0
new_accounts_opened_last_12months        0
multiple_applications_short_time_period  0
unusual_submission_pattern               0
applications_submitted_during_odd_hours  0
watchlist_blacklist_flag                 0
public_records_flag                      0
dtype: int64
```

In [117]:
```
df.dtypes
```

Out[117]:
```
account_open_date                           object
age                                          int64
location                                    object
occupation                                  object
income_level                                 int64
fico_score                                 float64
delinquency_status                           int64
charge_off_status                             bool
number_of_credit_applications                int64
debt_to_income_ratio                       float64
payment_methods_high_risk                     bool
max_balance                                float64
avg_balance_last_12months                  float64
number_of_delinquent_accounts              float64
number_of_defaulted_accounts                 int64
earliest_credit_account                     object
recent_trade_activity                       object
new_accounts_opened_last_12months            int64
multiple_applications_short_time_period       bool
unusual_submission_pattern                 float64
applications_submitted_during_odd_hours       bool
watchlist_blacklist_flag                      bool
public_records_flag                           bool
dtype: object
```

In [118]:
```
# Convert object columns into date type
date_columns = ['account_open_date', 'earliest_credit_account','recent_trade_activity'  ]
df[date_columns] = df[date_columns].apply(pd.to_datetime, errors='coerce')
```

In [119]:
```
df.dtypes
```

Out[119]:
```
account_open_date                      datetime64[ns]
age                                             int64
location                                       object
occupation                                     object
income_level                                    int64
fico_score                                    float64
delinquency_status                              int64
charge_off_status                                bool
number_of_credit_applications                   int64
debt_to_income_ratio                          float64
payment_methods_high_risk                        bool
max_balance                                   float64
avg_balance_last_12months                     float64
number_of_delinquent_accounts                 float64
number_of_defaulted_accounts                    int64
earliest_credit_account                datetime64[ns]
recent_trade_activity                  datetime64[ns]
new_accounts_opened_last_12months               int64
multiple_applications_short_time_period          bool
unusual_submission_pattern                    float64
applications_submitted_during_odd_hours          bool
watchlist_blacklist_flag                         bool
public_records_flag                              bool
dtype: object
```

In [120]:
```
df['unusual_submission_pattern'] = df['unusual_submission_pattern'].astype(bool)
```

In [121]:
```
df
```

Out[121]:

	account_open_date	age	location	occupation	income_level	fico_score	delinquency_status	charge_off_status	number_of_credit_applications	debt_
0	2023-11-09	56	Los Angeles	Teacher	40099	424.0	108	True	10	
1	2022-09-11	69	New York	Engineer	2050	483.0	0	False	1	
2	2020-07-12	46	Miami	Engineer	71936	566.0	0	False	1	
3	2024-08-13	32	Houston	Banker	15833	636.0	97	True	5	
4	2024-07-27	60	Houston	Lawyer	8574	787.0	0	False	1	
...	...	...	...	...	...	...	...	...	...	
6995	2021-04-10	33	Boston	Consultant	14566	589.0	92	True	2	
6996	2022-03-18	66	Atlanta	Banker	8930	679.0	0	True	1	
6997	2024-06-20	68	Houston	Other	4501	321.0	58	True	1	
6998	2020-10-04	35	Chicago	Retail Worker	11938	729.0	0	False	1	
6999	2020-06-13	52	Seattle	Banker	7861	359.0	103	True	5	

7000 rows × 23 columns

In [122]:
```
df.duplicated().sum()
```

Out[122]: 0

In [123]:
```
df
```

Out[123]:

	account_open_date	age	location	occupation	income_level	fico_score	delinquency_status	charge_off_status	number_of_credit_applications	debt_
0	2023-11-09	56	Los Angeles	Teacher	40099	424.0	108	True	10	
1	2022-09-11	69	New York	Engineer	2050	483.0	0	False	1	
2	2020-07-12	46	Miami	Engineer	71936	566.0	0	False	1	
3	2024-08-13	32	Houston	Banker	15833	636.0	97	True	5	
4	2024-07-27	60	Houston	Lawyer	8574	787.0	0	False	1	
...	...	...	...	...	...	...	...	...	...	
6995	2021-04-10	33	Boston	Consultant	14566	589.0	92	True	2	
6996	2022-03-18	66	Atlanta	Banker	8930	679.0	0	True	1	
6997	2024-06-20	68	Houston	Other	4501	321.0	58	True	1	
6998	2020-10-04	35	Chicago	Retail Worker	11938	729.0	0	False	1	
6999	2020-06-13	52	Seattle	Banker	7861	359.0	103	True	5	

7000 rows × 23 columns

In [124]:
```
df['date_contradiction_flag'] = (
    (df['earliest_credit_account'] > df['account_open_date']) |
    (df['account_open_date'] > df['recent_trade_activity']) |
    (df['earliest_credit_account'] > df['recent_trade_activity'])
)
```

In [125]:
```
df['date_contradiction_flag'].value_counts()
```

Out[125]:
```
False    4570
True     2430
Name: date_contradiction_flag, dtype: int64
```

In [126]:
```
contradiction_count = df['date_contradiction_flag'].sum()
print(f"Number of rows with date contradictions: {contradiction_count}")
```

```
Number of rows with date contradictions: 2430
```

In [127]:
```
df.loc[df['date_contradiction_flag'], 'earliest_credit_account'] = df[['earliest_credit_account', 'account_open_date', 'rece
df.loc[df['date_contradiction_flag'], 'recent_trade_activity'] = df[['earliest_credit_account', 'account_open_date', 'recent_
```

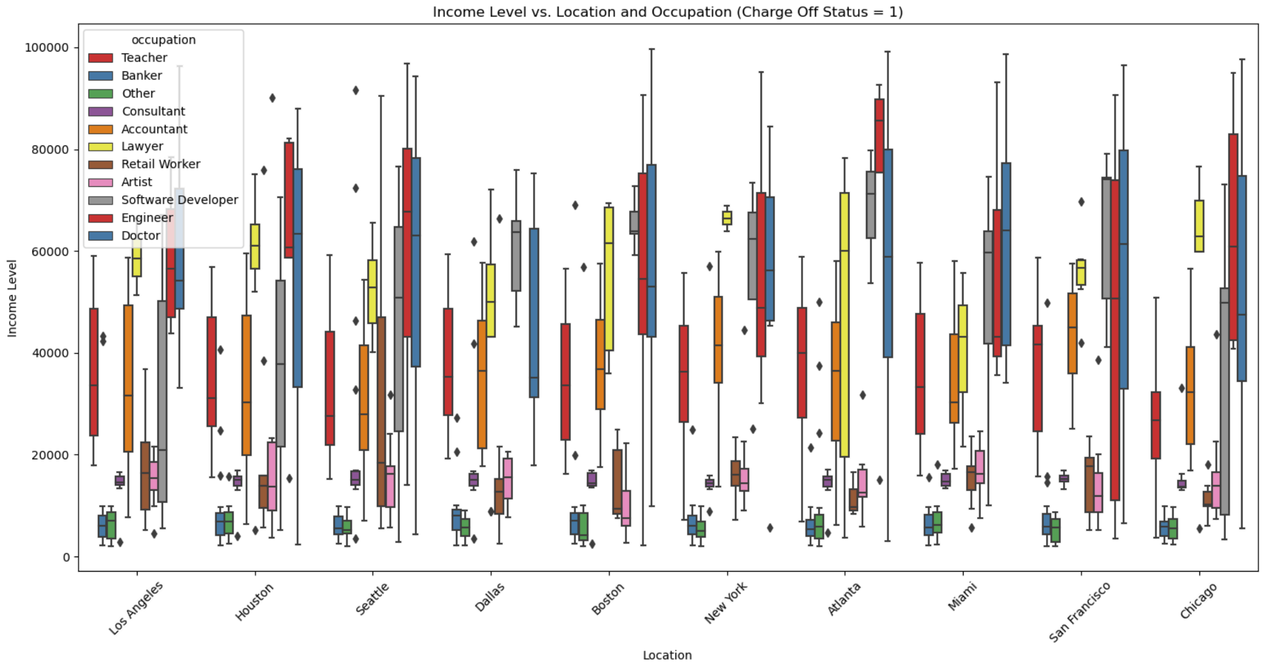
In [130]:
```
df
```

Out[130]:

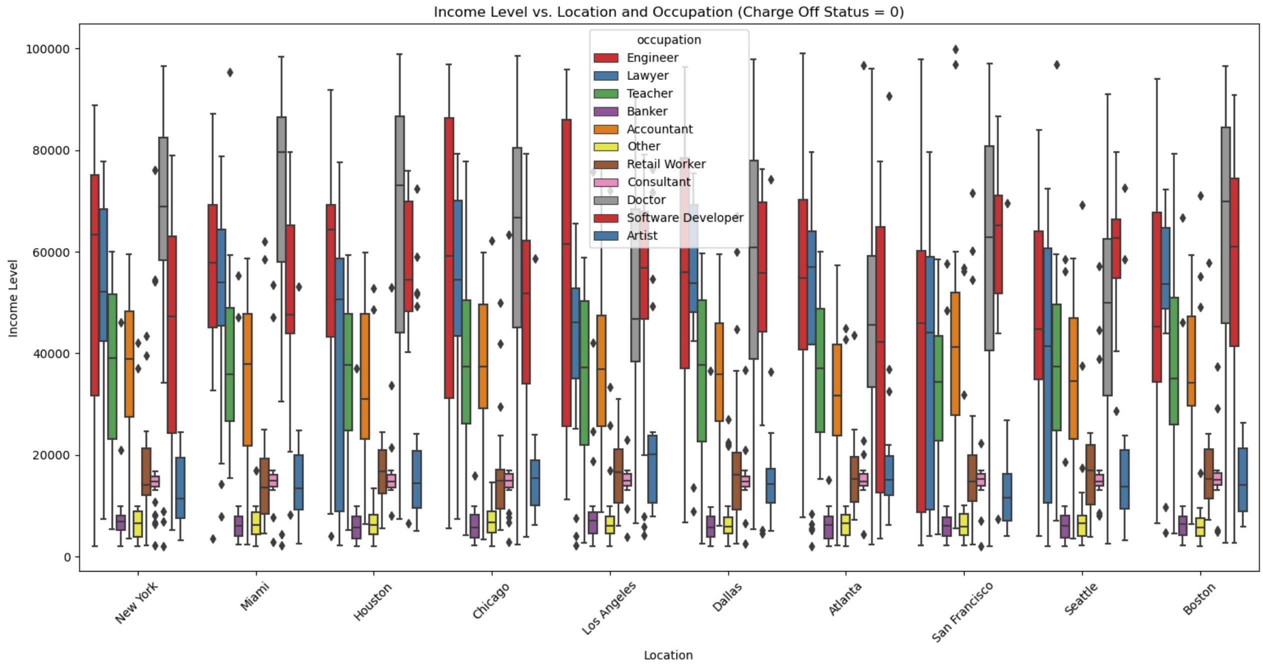
	account_open_date	age	location	occupation	income_level	fico_score	delinquency_status	charge_off_status	number_of_credit_applications	debt_
0	2023-11-09	56	Los Angeles	Teacher	40099	424.0	108	True	10	
1	2022-09-11	69	New York	Engineer	2050	483.0	0	False	1	
2	2020-07-12	46	Miami	Engineer	71936	566.0	0	False	1	
3	2024-08-13	32	Houston	Banker	15833	636.0	97	True	5	
4	2024-07-27	60	Houston	Lawyer	8574	787.0	0	False	1	
...	...	...	...	...	...	...	...	...	...	
6995	2021-04-10	33	Boston	Consultant	14566	589.0	92	True	2	
6996	2022-03-18	66	Atlanta	Banker	8930	679.0	0	True	1	
6997	2024-06-20	68	Houston	Other	4501	321.0	58	True	1	
6998	2020-10-04	35	Chicago	Retail Worker	11938	729.0	0	False	1	
6999	2020-06-13	52	Seattle	Banker	7861	359.0	103	True	5	

7000 rows × 24 columns

In [140]:
```python
import matplotlib.pyplot as plt
import seaborn as sns

# Filter the DataFrame to include only rows where charge_off_status is 1
df_charge_off_1 = df[df['charge_off_status'] == 1]


plt.figure(figsize=(15, 8))

# Boxplot: income vs. location and occupation, considering only charge_off_status = 1
sns.boxplot(x='location', y='income_level', data=df_charge_off_1, hue='occupation', palette='Set1')


plt.title('Income Level vs. Location and Occupation (Charge Off Status = 1)')
plt.ylabel('Income Level')
plt.xlabel('Location')
plt.xticks(rotation=45)


plt.tight_layout()


plt.show()
```

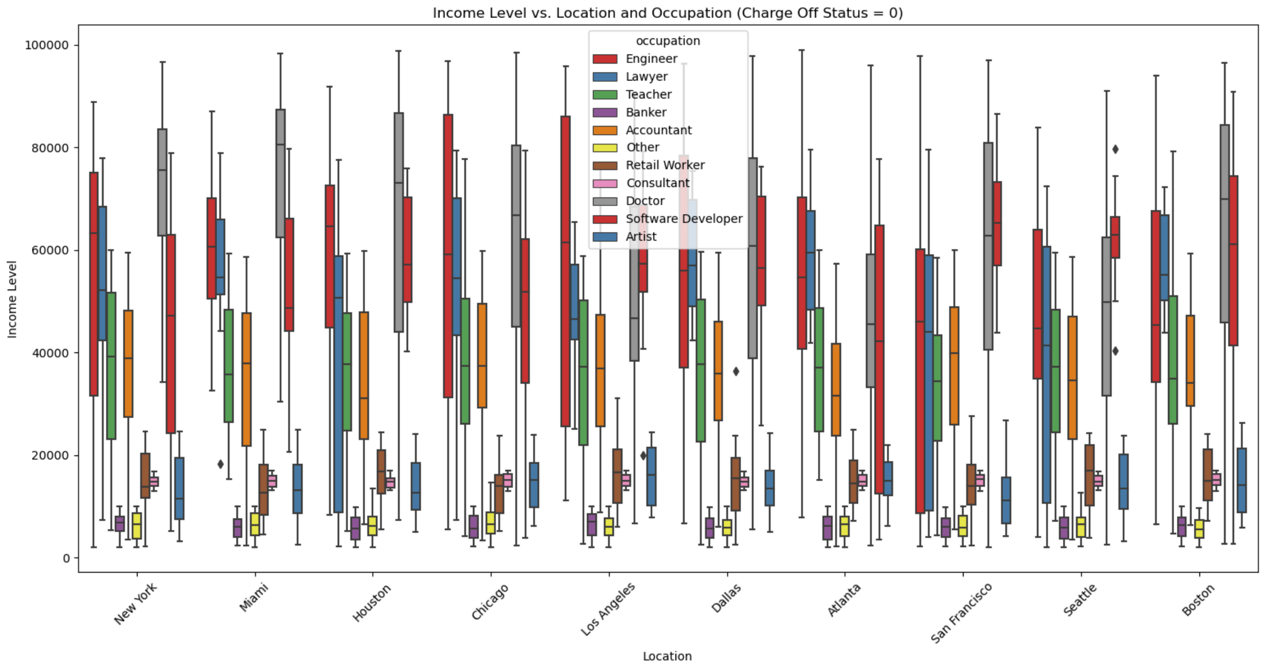


In [142]:
```
import matplotlib.pyplot as plt
import seaborn as sns

# Filter the DataFrame to include only rows where charge_off_status is 0
df_charge_off_0 = df[df['charge_off_status'] == 0]


plt.figure(figsize=(15, 8))

# Boxplot: income vs. location and occupation, considering only charge_off_status = 0
sns.boxplot(x='location', y='income_level', data=df_charge_off_0, hue='occupation', palette='Set1')


plt.title('Income Level vs. Location and Occupation (Charge Off Status = 0)')
plt.ylabel('Income Level')
plt.xlabel('Location')
plt.xticks(rotation=45)


plt.tight_layout()


plt.show()
```

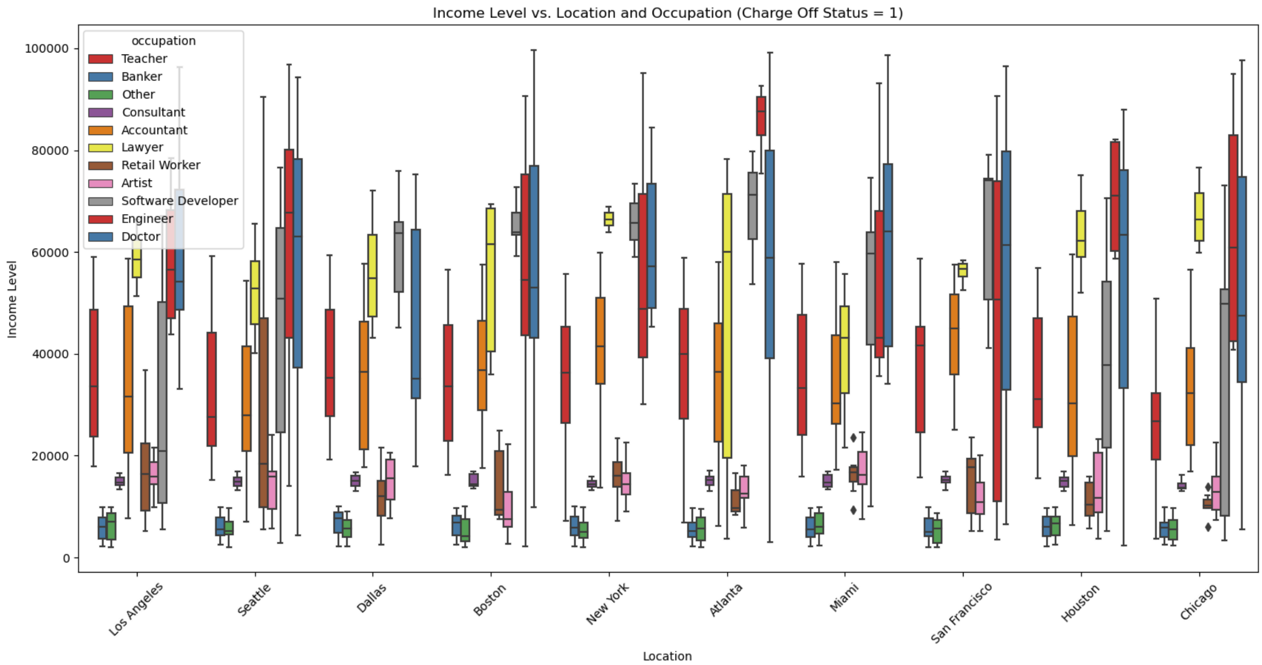


In [151]:
```
def remove_outliers_by_group(df, group_cols, value_col):

    Q1 = df.groupby(group_cols)[value_col].transform(lambda x: x.quantile(0.25))
    Q3 = df.groupby(group_cols)[value_col].transform(lambda x: x.quantile(0.75))
    IQR = Q3 - Q1


    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR


    df = df[(df[value_col] >= lower_bound) & (df[value_col] <= upper_bound)]
    return df


df_no_outliers_charge_off_0 = remove_outliers_by_group(df_charge_off_0, group_cols=['location', 'occupation'], value_col='in

df_no_outliers_charge_off_1 = remove_outliers_by_group(df_charge_off_1, group_cols=['location', 'occupation'], value_col='in
```

In [152]:
```
import matplotlib.pyplot as plt
import seaborn as sns



plt.figure(figsize=(15, 8))

# Boxplot: income vs. location and occupation, considering only charge_off_status = 0
sns.boxplot(x='location', y='income_level', data=df_no_outliers_charge_off_0, hue='occupation', palette='Set1')


plt.title('Income Level vs. Location and Occupation (Charge Off Status = 0)')
plt.ylabel('Income Level')
plt.xlabel('Location')
plt.xticks(rotation=45)


plt.tight_layout()


plt.show()
```



In [153]:
```
import matplotlib.pyplot as plt
import seaborn as sns



# Set the figure size
plt.figure(figsize=(15, 8))

# Boxplot: income vs. location and occupation, considering only charge_off_status = 1
sns.boxplot(x='location', y='income_level', data=df_no_outliers_charge_off_1, hue='occupation', palette='Set1')

# Add title and labels
plt.title('Income Level vs. Location and Occupation (Charge Off Status = 1)')
plt.ylabel('Income Level')
plt.xlabel('Location')
plt.xticks(rotation=45)

# Adjust layout for better display
plt.tight_layout()

# Show plot
plt.show()
```



In [164]:
```
df = pd.concat([df_no_outliers_charge_off_0 , df_no_outliers_charge_off_1] , axis=0)
```

In [165]:
```
df
```

Out[165]:

	account_open_date	age	location	occupation	income_level	fico_score	delinquency_status	charge_off_status	number_of_credit_applications	debt_
1	2022-09-11	69	New York	Engineer	2050	483.0	0	False	1	
2	2020-07-12	46	Miami	Engineer	71936	566.0	0	False	1	
4	2024-07-27	60	Houston	Lawyer	8574	787.0	0	False	1	
5	2022-03-06	25	Chicago	Teacher	59349	688.0	0	False	1	
6	2023-10-26	38	Los Angeles	Banker	6925	740.0	0	False	1	
...	...	...	...	...	...	...	...	...	...	
6990	2019-12-27	74	Chicago	Doctor	51012	580.0	93	True	3	
6995	2021-04-10	33	Boston	Consultant	14566	589.0	92	True	2	
6996	2022-03-18	66	Atlanta	Banker	8930	679.0	0	True	1	
6997	2024-06-20	68	Houston	Other	4501	321.0	58	True	1	
6999	2020-06-13	52	Seattle	Banker	7861	359.0	103	True	5	

6784 rows × 24 columns

In [167]:

```
outliers_age = df[((df['age'] - df['age'].mean()) / df['age'].std()).abs() > 3]
outliers_age
```

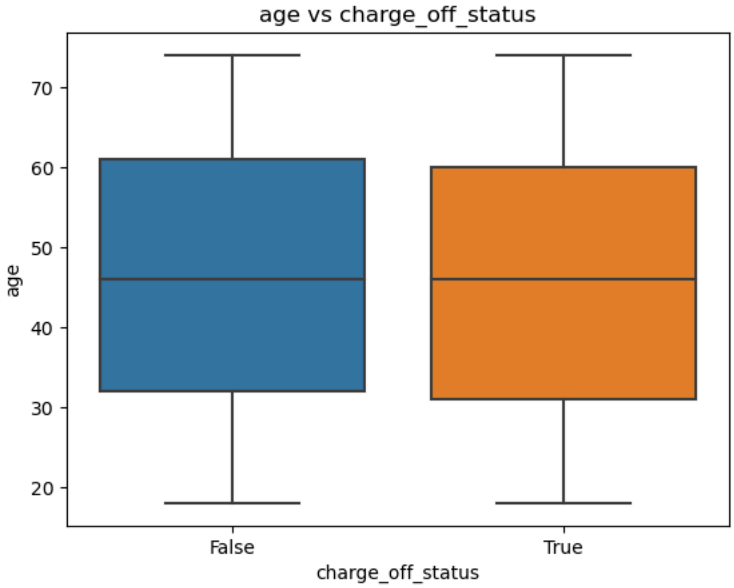
Out[167]:

account_open_date	age	location	occupation	income_level	fico_score	delinquency_status	charge_off_status	number_of_credit_applications	debt_to_ir

0 rows × 24 columns

In [168]:

```
sns.boxplot(x=df['charge_off_status'], y=df['age'])
plt.title('age vs charge_off_status')
plt.ylabel('age')
plt.xlabel('charge_off_status')
plt.show()
```



In [169]:
```
outliers_fico_score = df[(df['fico_score'] < 300) | (df['fico_score'] > 850)]
outliers_fico_score
```

Out[169]:

	account_open_date	age	location	occupation	income_level	fico_score	delinquency_status	charge_off_status	number_of_credit_applications	debt
862	2024-02-19	46	Miami	Banker	3202	938.0	0	False	2	
908	2021-03-25	48	Seattle	Accountant	39792	278.0	0	False	1	
920	2024-05-03	34	Boston	Other	7897	906.0	0	False	1	
1110	2024-03-18	43	Boston	Artist	5879	287.0	0	False	1	
1288	2021-06-29	40	Boston	Accountant	45827	931.0	0	False	1	
1989	2023-11-06	23	Seattle	Teacher	45887	940.0	0	False	1	
2050	2022-05-27	30	Houston	Other	7568	243.0	4	False	1	
2158	2024-09-28	36	Boston	Teacher	27337	916.0	0	False	1	
2287	2024-05-26	26	Los Angeles	Teacher	24534	174.0	0	False	1	
2893	2021-07-14	62	Seattle	Accountant	24097	234.0	25	False	1	
3549	2020-03-09	48	Atlanta	Other	5151	908.0	2	False	1	
3978	2023-10-24	52	New York	Accountant	22013	903.0	0	False	1	
4271	2022-04-26	41	Seattle	Banker	5753	241.0	0	False	1	
4294	2020-07-07	52	Seattle	Lawyer	64826	152.0	0	False	1	
4299	2024-06-15	26	San Francisco	Doctor	49055	901.0	0	False	1	
4466	2023-05-05	74	Seattle	Accountant	12278	913.0	0	False	1	
4723	2022-04-13	24	Miami	Banker	4270	255.0	0	False	1	
4867	2023-09-21	43	Boston	Lawyer	72255	196.0	0	False	1	
5202	2024-01-18	38	Dallas	Other	5610	909.0	0	False	1	
5296	2024-08-13	66	Houston	Consultant	13826	183.0	0	False	1	
5524	2020-05-31	58	New York	Engineer	67105	914.0	0	False	1	
5641	2019-10-31	52	New York	Teacher	42383	940.0	0	False	1	
6018	2024-05-21	28	Chicago	Accountant	31640	936.0	0	False	1	
6376	2024-05-21	70	Miami	Accountant	19168	900.0	0	False	1	
6549	2024-01-20	36	New York	Banker	7661	902.0	1	False	1	
6673	2024-05-30	61	Boston	Other	8028	217.0	0	False	1	
6729	2020-07-23	51	Miami	Teacher	56174	279.0	0	False	1	
131	2022-02-26	24	Miami	Software Developer	9972	920.0	98	True	6	
161	2024-08-11	26	Houston	Other	6959	923.0	104	True	2	
2219	2023-06-17	34	Miami	Doctor	78377	913.0	111	True	2	
2671	2023-09-02	40	New York	Banker	3133	205.0	99	True	4	
2779	2022-05-11	52	Los Angeles	Teacher	21634	920.0	0	True	1	
3419	2024-05-31	72	Houston	Teacher	56837	922.0	0	True	1	
3921	2023-06-28	64	Miami	Other	8846	924.0	106	True	4	
4087	2022-08-22	56	Seattle	Teacher	25870	943.0	104	True	1	
4157	2023-05-16	68	Boston	Banker	3732	915.0	99	True	10	
4811	2023-12-04	33	Chicago	Accountant	22595	902.0	90	True	2	
5640	2020-07-02	36	Miami	Teacher	47439	913.0	0	True	1	
6567	2020-08-18	49	Atlanta	Teacher	42774	154.0	0	True	1	
6591	2021-05-08	55	Atlanta	Banker	2335	200.0	0	True	1	
6681	2023-03-28	52	Seattle	Other	2281	918.0	0	True	1	

41 rows × 24 columns

In [170]:
```
# Remove fico_score outliers from the dataset
df = df[(df['fico_score'] >= 300) & (df['fico_score'] <= 850)]


df.reset_index(drop=True, inplace=True)

# Display the dataset without fico_score outliers
df
```

Out[170]:

	account_open_date	age	location	occupation	income_level	fico_score	delinquency_status	charge_off_status	number_of_credit_applications	debt_
0	2022-09-11	69	New York	Engineer	2050	483.0	0	False	1	
1	2020-07-12	46	Miami	Engineer	71936	566.0	0	False	1	
2	2024-07-27	60	Houston	Lawyer	8574	787.0	0	False	1	
3	2022-03-06	25	Chicago	Teacher	59349	688.0	0	False	1	
4	2023-10-26	38	Los Angeles	Banker	6925	740.0	0	False	1	
...	...	...	...	...	...	...	...	...	...	
6738	2019-12-27	74	Chicago	Doctor	51012	580.0	93	True	3	
6739	2021-04-10	33	Boston	Consultant	14566	589.0	92	True	2	
6740	2022-03-18	66	Atlanta	Banker	8930	679.0	0	True	1	
6741	2024-06-20	68	Houston	Other	4501	321.0	58	True	1	
6742	2020-06-13	52	Seattle	Banker	7861	359.0	103	True	5	

6743 rows × 24 columns

In [171]:
```
outliers_fico_score = df[(df['fico_score'] < 300) | (df['fico_score'] > 850)]
outliers_fico_score
```

Out[171]:

account_open_date	age	location	occupation	income_level	fico_score	delinquency_status	charge_off_status	number_of_credit_applications	debt_to_ir

0 rows × 24 columns

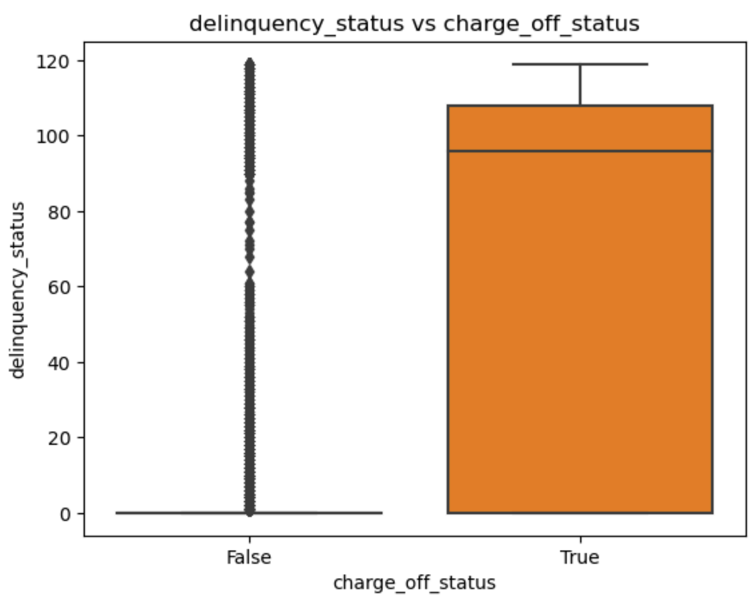
In [172]:
```

outliers_delinquency_status = df[((df['delinquency_status'] - df['delinquency_status'].mean()) / df['delinquency_status'].st
outliers_delinquency_status
```

Out[172]:

account_open_date	age	location	occupation	income_level	fico_score	delinquency_status	charge_off_status	number_of_credit_applications	debt_to_ir

0 rows × 24 columns

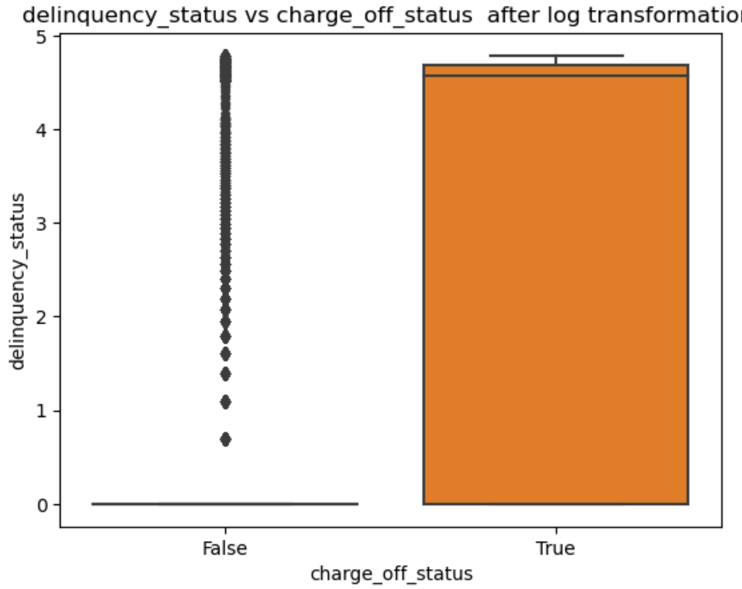
In [173]:
```
sns.boxplot(x=df['charge_off_status'], y=df['delinquency_status'])
plt.title('delinquency_status vs charge_off_status')
plt.ylabel('delinquency_status')
plt.xlabel('charge_off_status')
plt.show()
```



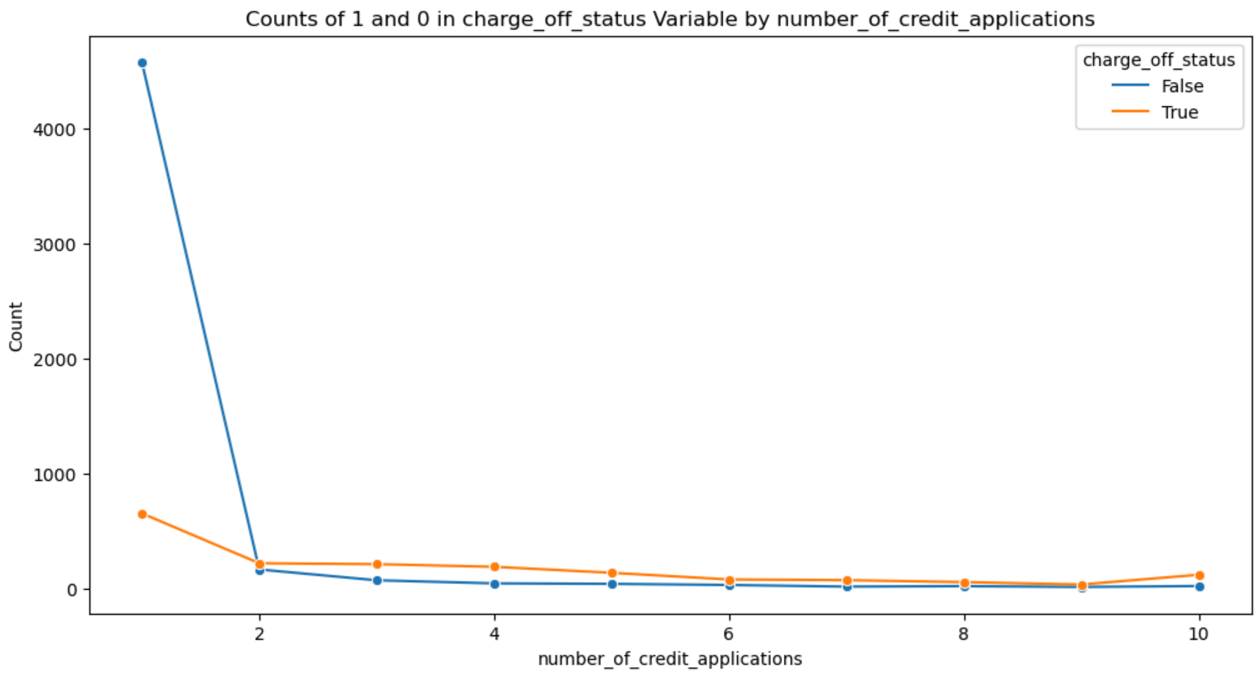
In [174]:
```
# Apply a log transformation
df['delinquency_status_log'] = np.log1p(df['delinquency_status'])
```

```
C:\Users\THUSHAN\AppData\Local\Temp\ipykernel_6948\3838396041.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-vie
w-versus-a-copy (https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)
  df['delinquency_status_log'] = np.log1p(df['delinquency_status'])
```

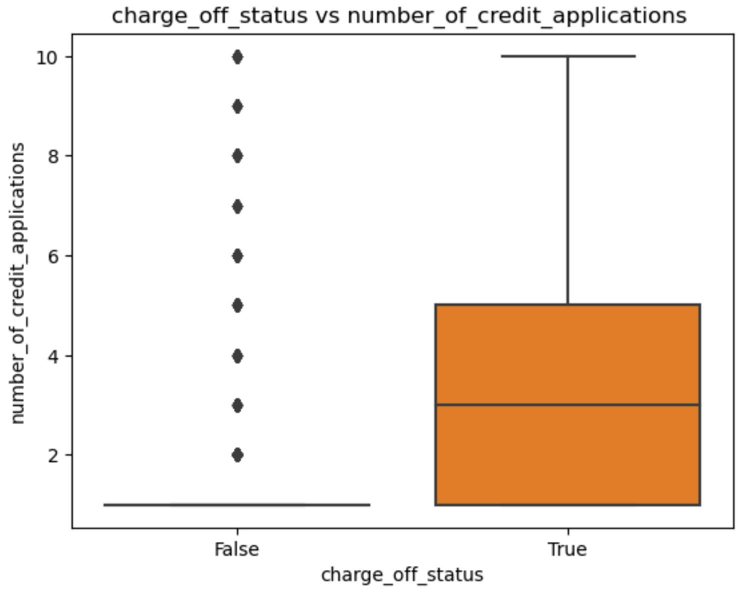
In [175]:
```
sns.boxplot(x=df['charge_off_status'], y=df['delinquency_status_log'])
plt.title('delinquency_status vs charge_off_status  after log transformation ')
plt.ylabel('delinquency_status')
plt.xlabel('charge_off_status')
plt.show()
```



In [176]:
```
# Group by date and accept, then count occurrences
grouped_df = df.groupby(['number_of_credit_applications', 'charge_off_status']).size().reset_index(name='count')

# Plot
plt.figure(figsize=(12, 6))
sns.lineplot(data=grouped_df, x='number_of_credit_applications', y='count', hue='charge_off_status', marker='o')
plt.title('Counts of 1 and 0 in charge_off_status Variable by number_of_credit_applications')
plt.xlabel('number_of_credit_applications')
plt.ylabel('Count')
plt.legend(title='charge_off_status')
plt.show()
```



In [177]:
```
sns.boxplot(x=df['charge_off_status'], y=df['number_of_credit_applications'])
plt.title('charge_off_status vs number_of_credit_applications ')
plt.xlabel('charge_off_status')
plt.ylabel('number_of_credit_applications')
plt.show()
```



In [178]:
```
# see how many ouliers in charge_off_status = 0 & number_of_credit_applications > 1
filtered_charge_off_status_outliers = df[(df['charge_off_status'] == 0) & (df['number_of_credit_applications'] > 1)]
filtered_charge_off_status_outliers
```

Out[178]:

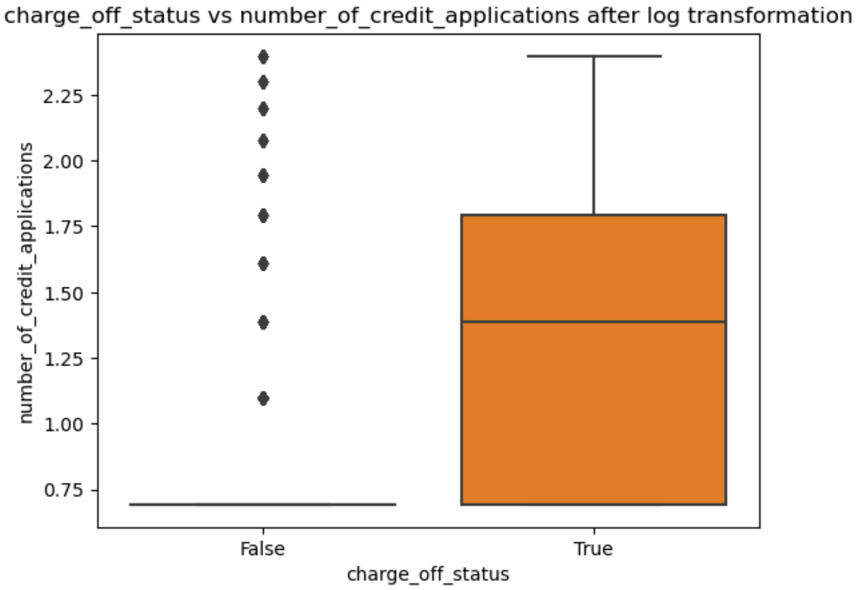
	account_open_date	age	location	occupation	income_level	fico_score	delinquency_status	charge_off_status	number_of_credit_applications	debt_
10	2023-03-19	53	Houston	Accountant	27455	470.0	105	False	8	
16	2021-04-04	41	Dallas	Software Developer	72993	672.0	39	False	2	
26	2020-02-09	44	Houston	Accountant	31500	310.0	90	False	6	
34	2021-04-15	24	Atlanta	Teacher	59938	433.0	106	False	3	
40	2024-03-24	31	Houston	Other	5598	636.0	0	False	2	
...	...	...	...	...	...	...	...	...	...	
4918	2024-03-31	26	Boston	Other	9199	526.0	60	False	2	
4942	2019-11-25	30	Boston	Teacher	34390	459.0	113	False	5	
4953	2024-01-11	69	Seattle	Other	8494	514.0	33	False	2	
4973	2021-07-26	52	Houston	Banker	2683	645.0	46	False	4	
4980	2020-12-21	72	Seattle	Doctor	71522	764.0	0	False	2	

411 rows × 25 columns

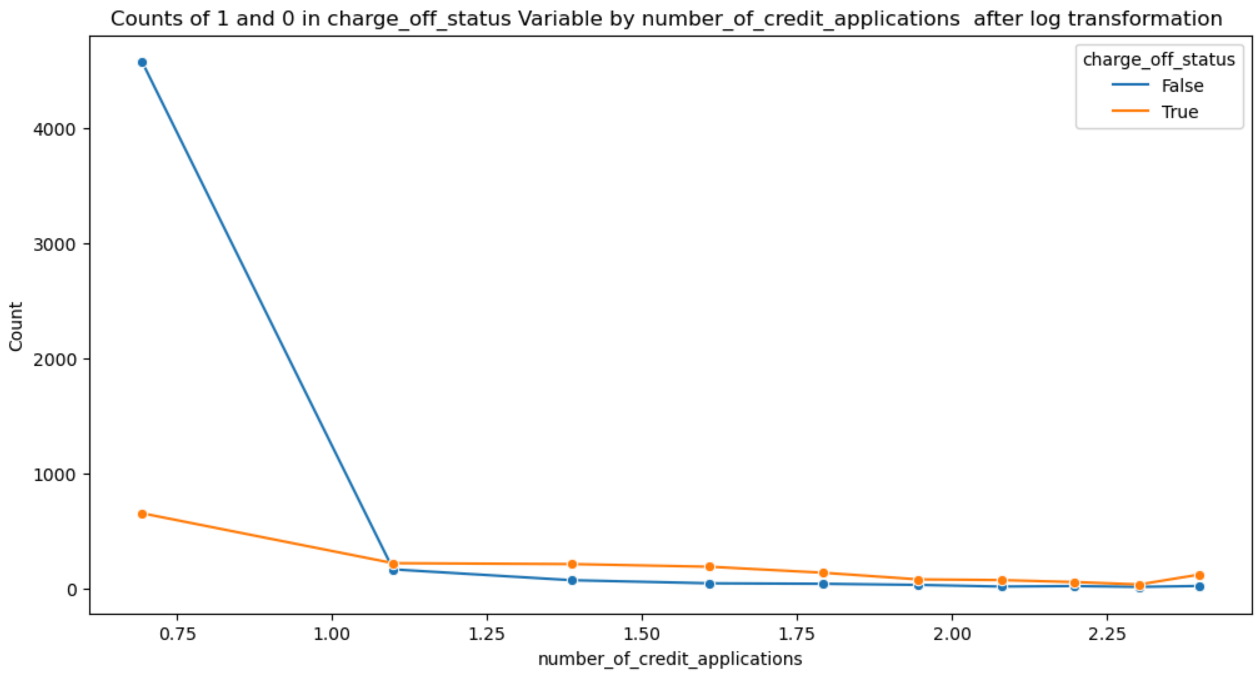
In [179]:
```
# Apply a log transformation
df['number_of_credit_applications_log'] = np.log1p(df['number_of_credit_applications'])
```

```
C:\Users\THUSHAN\AppData\Local\Temp\ipykernel_6948\1009209018.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy (https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)
  df['number_of_credit_applications_log'] = np.log1p(df['number_of_credit_applications'])
```

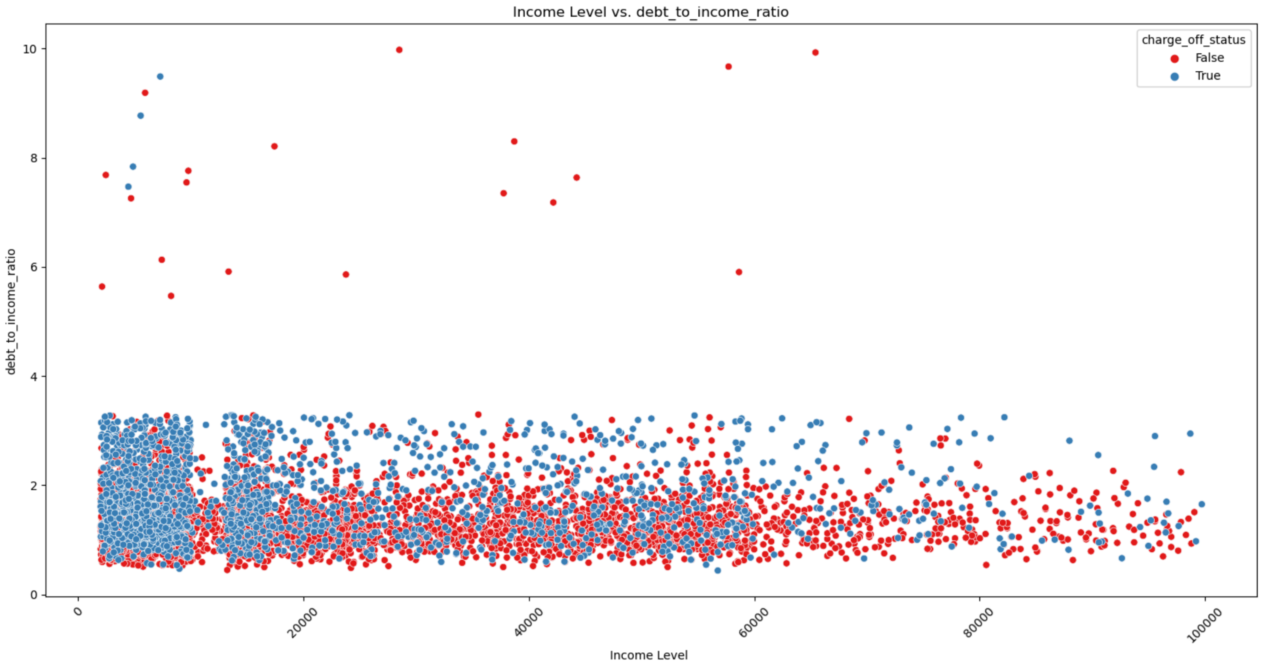
In [180]:
```
sns.boxplot(x=df['charge_off_status'], y=df['number_of_credit_applications_log'])
plt.title('charge_off_status vs number_of_credit_applications after log transformation')
plt.xlabel('charge_off_status')
plt.ylabel('number_of_credit_applications')
plt.show()
```



In [181]:
```
# Group by date and accept, then count occurrences
grouped_df = df.groupby(['number_of_credit_applications_log', 'charge_off_status']).size().reset_index(name='count')

# Plot
plt.figure(figsize=(12, 6))
sns.lineplot(data=grouped_df, x='number_of_credit_applications_log', y='count', hue='charge_off_status', marker='o')
plt.title('Counts of 1 and 0 in charge_off_status Variable by number_of_credit_applications  after log transformation ')
plt.xlabel('number_of_credit_applications')
plt.ylabel('Count')
plt.legend(title='charge_off_status')
plt.show()
```



In [182]:
```
# Boxplot: income vs. location and occupation
plt.figure(figsize=(15, 8))

sns.scatterplot(x='income_level', y='debt_to_income_ratio', data=df, hue='charge_off_status', palette='Set1')

plt.title('Income Level vs. debt_to_income_ratio')
plt.xlabel('Income Level ')
plt.ylabel('debt_to_income_ratio')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



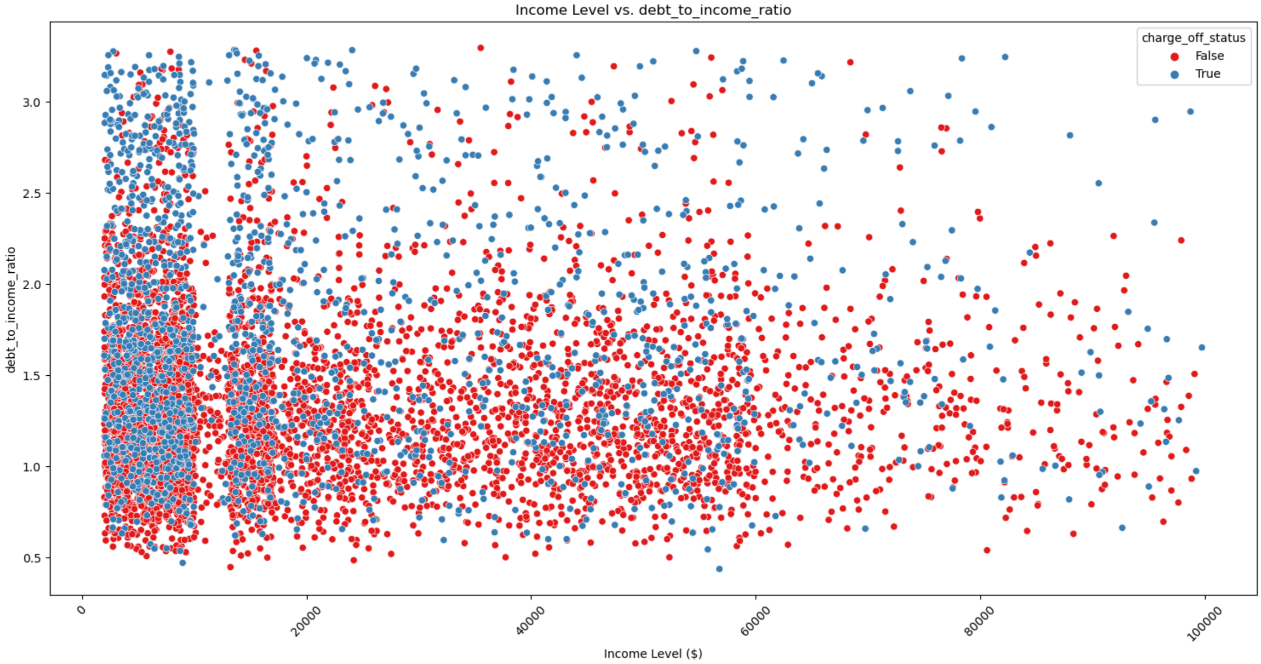
In [183]:
```
# Remove outliers from the dataset
df = df[(df['debt_to_income_ratio'] <= 4) ]


df.reset_index(drop=True, inplace=True)

# Display the dataset without outliers
df
```

Out[183]:

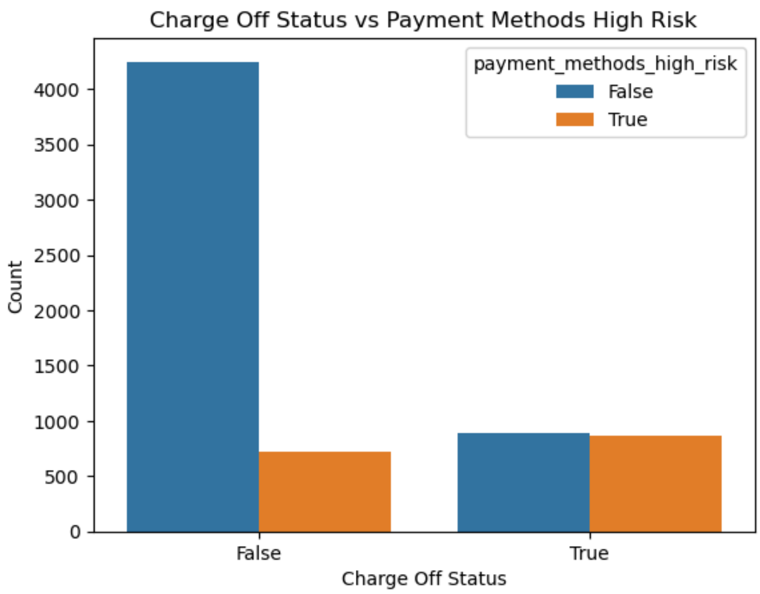
	account_open_date	age	location	occupation	income_level	fico_score	delinquency_status	charge_off_status	number_of_credit_applications	debt_
0	2022-09-11	69	New York	Engineer	2050	483.0	0	False	1	
1	2020-07-12	46	Miami	Engineer	71936	566.0	0	False	1	
2	2024-07-27	60	Houston	Lawyer	8574	787.0	0	False	1	
3	2022-03-06	25	Chicago	Teacher	59349	688.0	0	False	1	
4	2023-10-26	38	Los Angeles	Banker	6925	740.0	0	False	1	
...	...	...	...	...	...	...	...	...	...	
6715	2019-12-27	74	Chicago	Doctor	51012	580.0	93	True	3	
6716	2021-04-10	33	Boston	Consultant	14566	589.0	92	True	2	
6717	2022-03-18	66	Atlanta	Banker	8930	679.0	0	True	1	
6718	2024-06-20	68	Houston	Other	4501	321.0	58	True	1	
6719	2020-06-13	52	Seattle	Banker	7861	359.0	103	True	5	

6720 rows × 26 columns

In [184]:
```
# Boxplot: income vs. location and occupation
plt.figure(figsize=(15, 8))
# Plot income vs. location (you can also combine occupation here as hue)
sns.scatterplot(x='income_level', y='debt_to_income_ratio', data=df, hue='charge_off_status', palette='Set1')
# Customize plot
plt.title('Income Level vs. debt_to_income_ratio')
plt.xlabel('Income Level ($)')
plt.ylabel('debt_to_income_ratio')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



In [185]:
```
sns.countplot(x='charge_off_status', hue='payment_methods_high_risk', data=df)
plt.title('Charge Off Status vs Payment Methods High Risk')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.show()
```



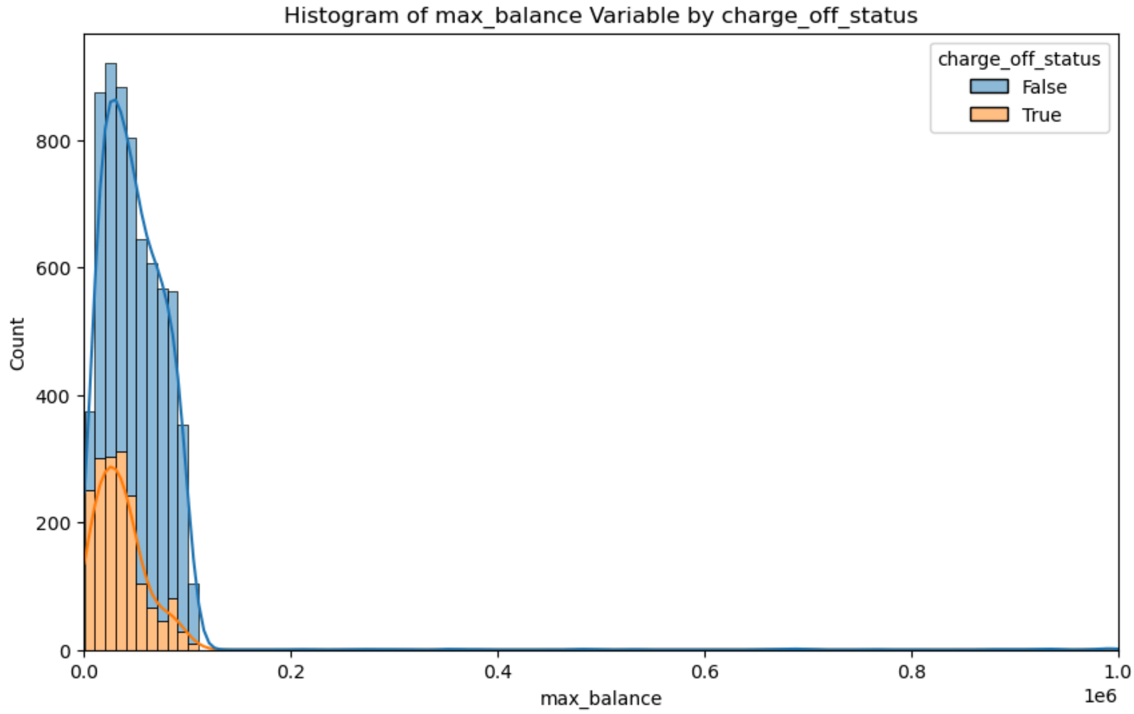
In [186]:
```
outliers_max_balance = df[((df['max_balance'] - df['max_balance'].mean()) / df['max_balance'].std()).abs() > 3]
outliers_max_balance
```

Out[186]:

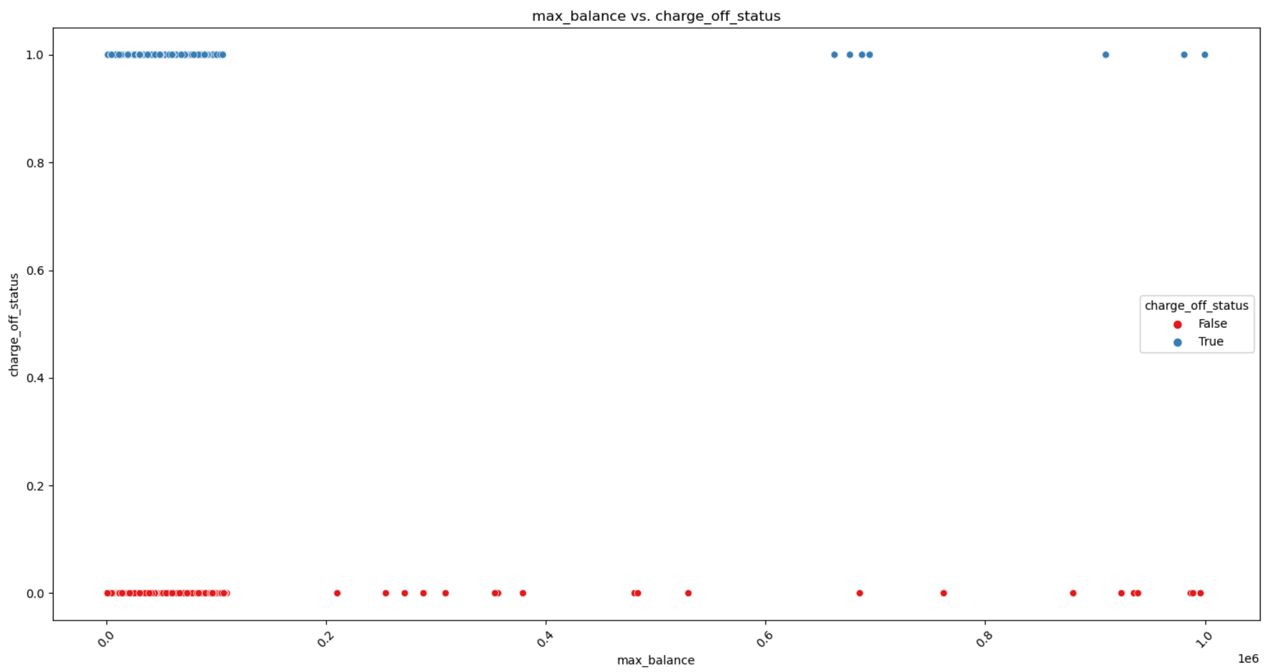
	account_open_date	age	location	occupation	income_level	fico_score	delinquency_status	charge_off_status	number_of_credit_applications	debt
102	2019-11-24	54	Chicago	Teacher	44367	678.0	0	False	1	
151	2022-10-05	23	Boston	Accountant	46852	465.0	1	False	1	
159	2021-08-29	58	Atlanta	Consultant	15103	777.0	0	False	1	
181	2024-04-25	43	Atlanta	Banker	7451	587.0	0	False	1	
303	2021-07-20	35	San Francisco	Banker	5669	626.0	0	False	1	
976	2022-05-02	22	Miami	Teacher	52207	328.0	97	False	5	
1086	2020-02-04	33	Atlanta	Software Developer	44512	506.0	0	False	1	
1111	2023-06-06	40	Dallas	Doctor	36557	652.0	0	False	1	
1208	2024-07-26	70	Los Angeles	Banker	7532	834.0	0	False	1	
1231	2024-08-05	51	Boston	Other	2218	791.0	0	False	1	
1853	2024-05-19	71	Dallas	Software Developer	25819	511.0	0	False	1	
2113	2023-04-18	50	Seattle	Artist	12167	733.0	0	False	1	
2190	2021-07-14	40	Houston	Other	8054	516.0	0	False	1	
3791	2023-09-27	58	Atlanta	Doctor	82260	713.0	0	False	1	
3875	2022-04-17	24	Boston	Teacher	16769	748.0	0	False	1	
4234	2023-07-27	49	Los Angeles	Teacher	34366	779.0	0	False	1	
4277	2022-09-15	32	Boston	Accountant	53358	452.0	0	False	1	
4703	2021-06-19	22	New York	Accountant	54643	713.0	0	False	1	
4857	2022-05-14	62	Dallas	Teacher	26838	810.0	0	False	1	
4894	2021-08-13	59	Los Angeles	Other	4241	653.0	0	False	1	
5063	2024-03-27	66	Atlanta	Teacher	26110	521.0	108	True	4	
5122	2022-11-11	27	Boston	Other	2015	364.0	108	True	2	
5316	2024-08-05	60	Dallas	Software Developer	45081	322.0	110	True	5	
5407	2023-07-16	42	New York	Banker	2112	521.0	92	True	2	
6098	2022-09-08	28	Seattle	Software Developer	22652	571.0	112	True	8	
6115	2023-09-17	21	San Francisco	Accountant	57542	734.0	0	True	1	
6701	2024-09-25	40	Los Angeles	Banker	3108	391.0	109	True	3	

27 rows × 26 columns

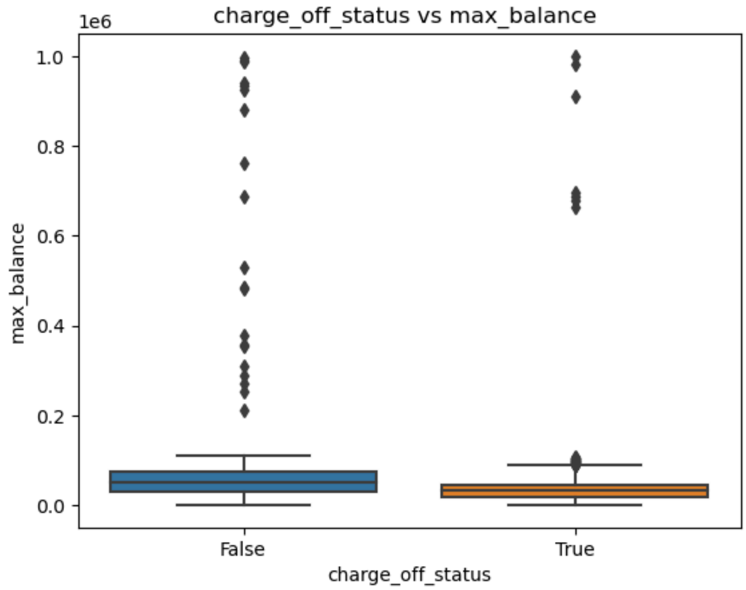
In [187]:
```
plt.figure(figsize=(10, 6))
sns.histplot(data=df, x='max_balance',  hue='charge_off_status', multiple='stack',bins=100, kde=True)
plt.xlabel('max_balance')
plt.xlim(0, 1000000)
plt.title('Histogram of max_balance Variable by charge_off_status')
plt.show()
```



In [188]:
```
# Boxplot: income vs. location and occupation
plt.figure(figsize=(15, 8))

sns.scatterplot(x='max_balance', y='charge_off_status', data=df, hue='charge_off_status', palette='Set1')

plt.title('max_balance vs. charge_off_status')
plt.xlabel('max_balance')
plt.ylabel('charge_off_status')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



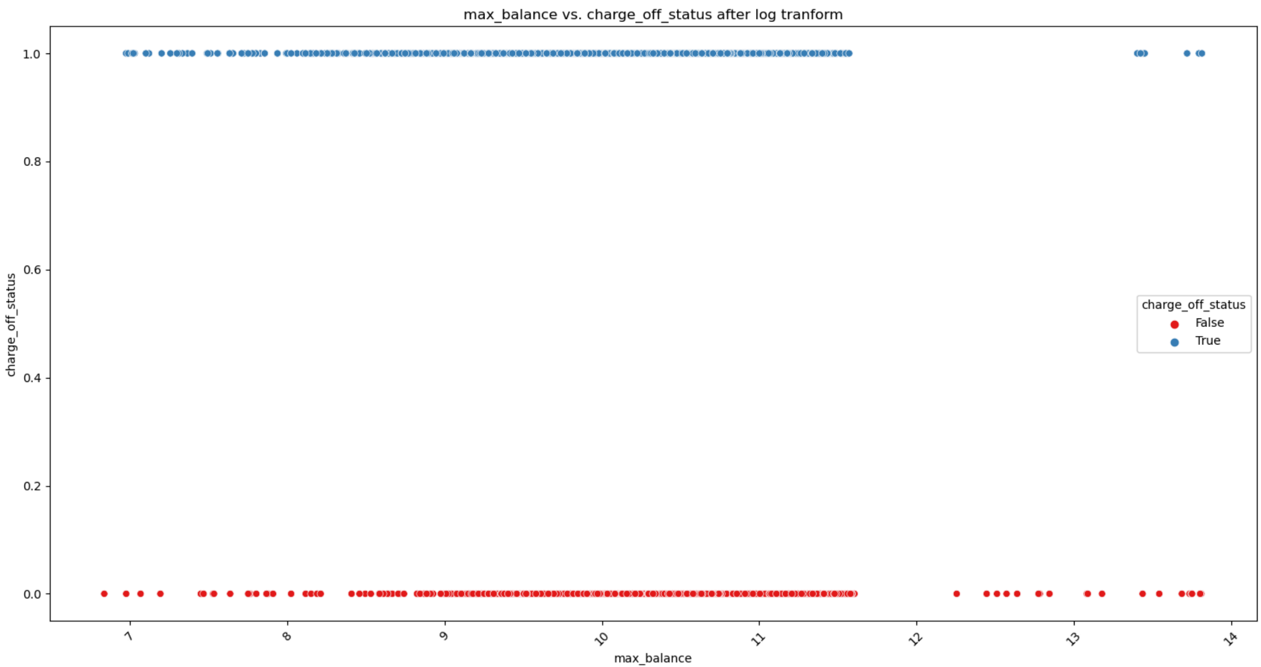
In [189]:
```
sns.boxplot(x=df['charge_off_status'], y=df['max_balance'])
plt.title('charge_off_status vs max_balance ')
plt.xlabel('charge_off_status')

plt.show()
```



In [190]:
```
# Apply a log transformation
df['max_balance_log'] = np.log1p(df['max_balance'])
```

```
C:\Users\THUSHAN\AppData\Local\Temp\ipykernel_6948\2433158796.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-vie
w-versus-a-copy (https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)
  df['max_balance_log'] = np.log1p(df['max_balance'])
```

In [191]:
```
# Boxplot: income vs. location and occupation
plt.figure(figsize=(15, 8))

sns.scatterplot(x='max_balance_log', y='charge_off_status', data=df, hue='charge_off_status', palette='Set1')
# Customize plot
plt.title('max_balance vs. charge_off_status after log tranform')
plt.xlabel('max_balance')
plt.ylabel('charge_off_status')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



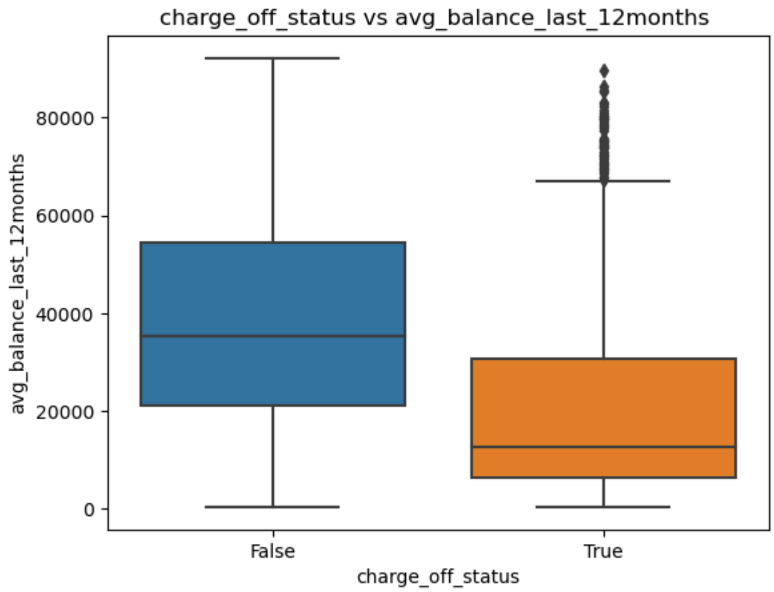
In [192]:
```
outliers_avg_balance_last_12months = df[((df['avg_balance_last_12months'] - df['avg_balance_last_12months'].mean()) / df['av
outliers_avg_balance_last_12months
```

Out[192]:

account_open_date	age	location	occupation	income_level	fico_score	delinquency_status	charge_off_status	number_of_credit_applications	debt_to_ir

0 rows × 27 columns

In [193]:
```
sns.boxplot(x=df['charge_off_status'], y=df['avg_balance_last_12months'])
plt.title('charge_off_status vs avg_balance_last_12months ')
plt.xlabel('charge_off_status')

plt.show()
```



In [194]:
```
# Filter data based on charge_off_status == 0
df_charge_off_0 = df[df['charge_off_status'] == 0]
df_charge_off_1 = df[df['charge_off_status'] == 1]

cap_value_0 = df_charge_off_0['avg_balance_last_12months'].quantile(0.95)
cap_value_1 = df_charge_off_1['avg_balance_last_12months'].quantile(0.95)

# Apply capping
df_charge_off_0['avg_balance_last_12months_capped'] = np.where(df_charge_off_0['avg_balance_last_12months'] > cap_value_0, ca
df_charge_off_1['avg_balance_last_12months_capped'] = np.where(df_charge_off_1['avg_balance_last_12months'] > cap_value_1, ca


# Concatenate the filtered dataframes
df = pd.concat([df_charge_off_0, df_charge_off_1])


df = df.reset_index(drop=True)


df
```

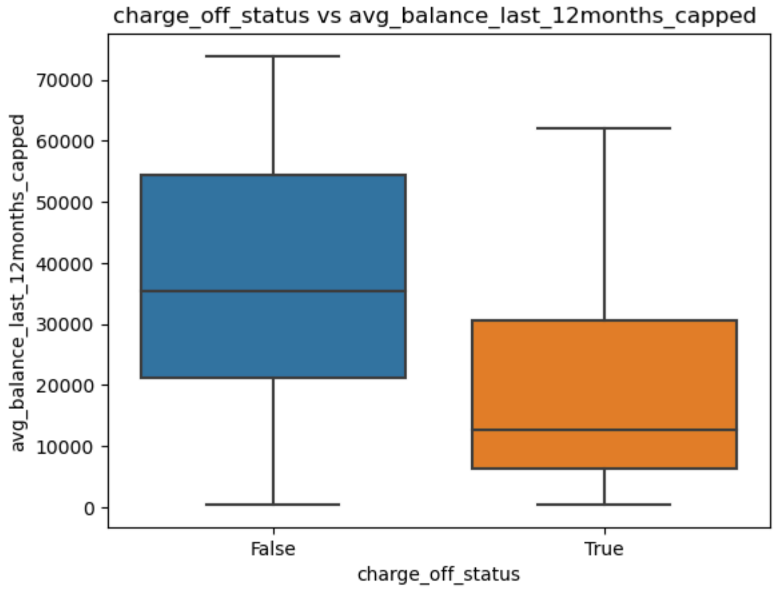
```
C:\Users\THUSHAN\AppData\Local\Temp\ipykernel_6948\4144690839.py:9: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-vie
w-versus-a-copy (https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)
  df_charge_off_0['avg_balance_last_12months_capped'] = np.where(df_charge_off_0['avg_balance_last_12months'] > cap_value_
0, cap_value_0, df_charge_off_0['avg_balance_last_12months'])
C:\Users\THUSHAN\AppData\Local\Temp\ipykernel_6948\4144690839.py:10: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-vie
w-versus-a-copy (https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)
  df_charge_off_1['avg_balance_last_12months_capped'] = np.where(df_charge_off_1['avg_balance_last_12months'] > cap_value_
1, cap_value_1, df_charge_off_1['avg_balance_last_12months'])
```

Out[194]:

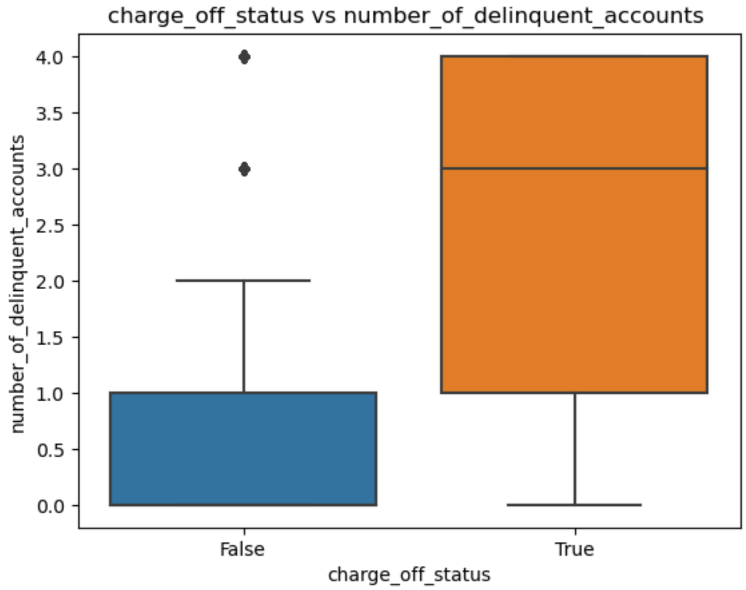
	account_open_date	age	location	occupation	income_level	fico_score	delinquency_status	charge_off_status	number_of_credit_applications	debt_
0	2022-09-11	69	New York	Engineer	2050	483.0	0	False	1	
1	2020-07-12	46	Miami	Engineer	71936	566.0	0	False	1	
2	2024-07-27	60	Houston	Lawyer	8574	787.0	0	False	1	
3	2022-03-06	25	Chicago	Teacher	59349	688.0	0	False	1	
4	2023-10-26	38	Los Angeles	Banker	6925	740.0	0	False	1	
...	...	...	...	...	...	...	...	...	...	
6715	2019-12-27	74	Chicago	Doctor	51012	580.0	93	True	3	
6716	2021-04-10	33	Boston	Consultant	14566	589.0	92	True	2	
6717	2022-03-18	66	Atlanta	Banker	8930	679.0	0	True	1	
6718	2024-06-20	68	Houston	Other	4501	321.0	58	True	1	
6719	2020-06-13	52	Seattle	Banker	7861	359.0	103	True	5	

6720 rows × 28 columns

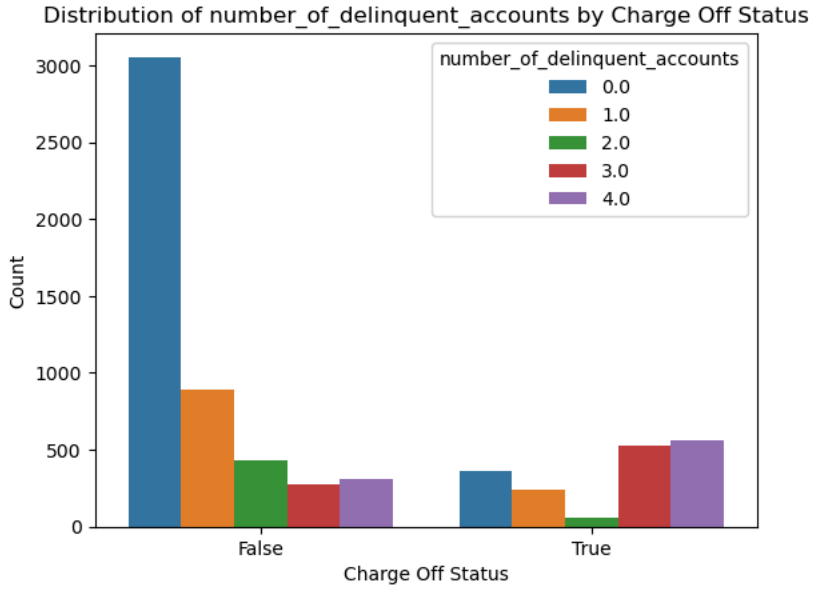
In [195]:
```
sns.boxplot(x=df['charge_off_status'], y=df['avg_balance_last_12months_capped'])
plt.title('charge_off_status vs avg_balance_last_12months_capped ')
plt.xlabel('charge_off_status')

plt.show()
```



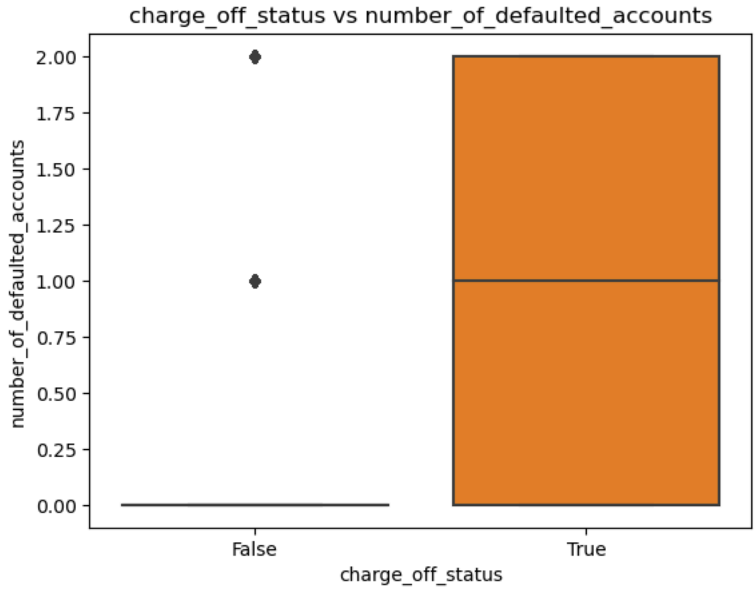
In [196]:
```
sns.boxplot(x=df['charge_off_status'], y=df['number_of_delinquent_accounts'])
plt.title('charge_off_status vs number_of_delinquent_accounts ')
plt.xlabel('charge_off_status')

plt.show()
```



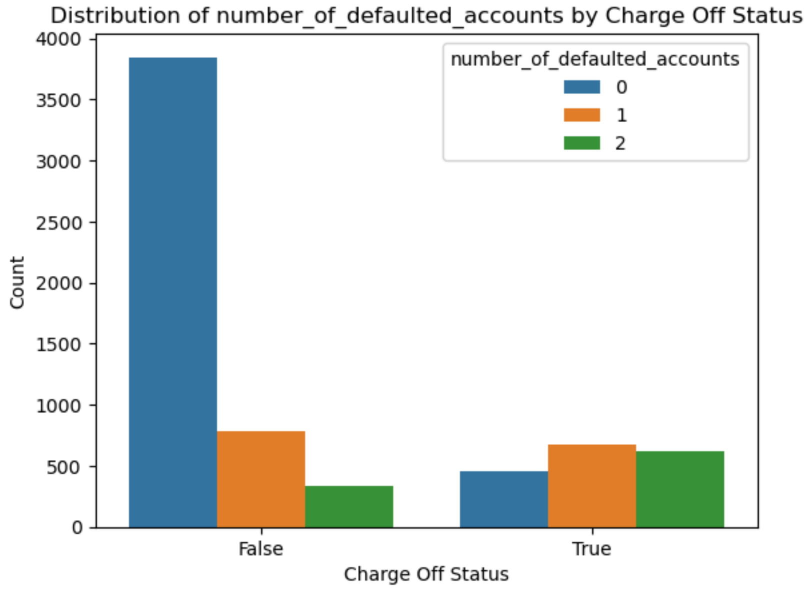
In [197]:
```
sns.countplot(x='charge_off_status', hue='number_of_delinquent_accounts', data=df)
plt.title('Distribution of number_of_delinquent_accounts by Charge Off Status')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.show()
```



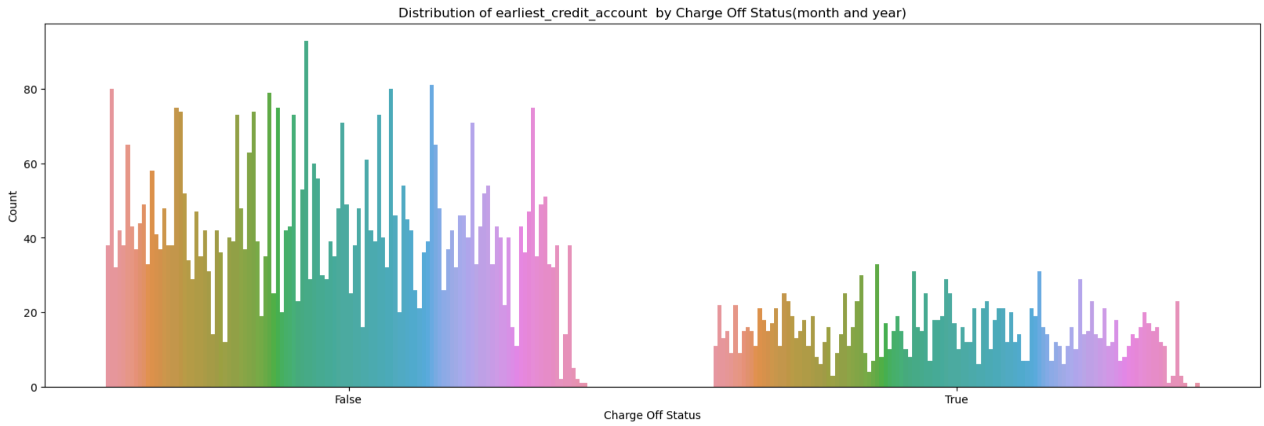
In [198]:
```
sns.boxplot(x=df['charge_off_status'], y=df['number_of_defaulted_accounts'])
plt.title('charge_off_status vs number_of_defaulted_accounts')
plt.xlabel('charge_off_status')

plt.show()
```



In [199]:
```
sns.countplot(x='charge_off_status', hue='number_of_defaulted_accounts', data=df)
plt.title('Distribution of number_of_defaulted_accounts by Charge Off Status')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.show()
```

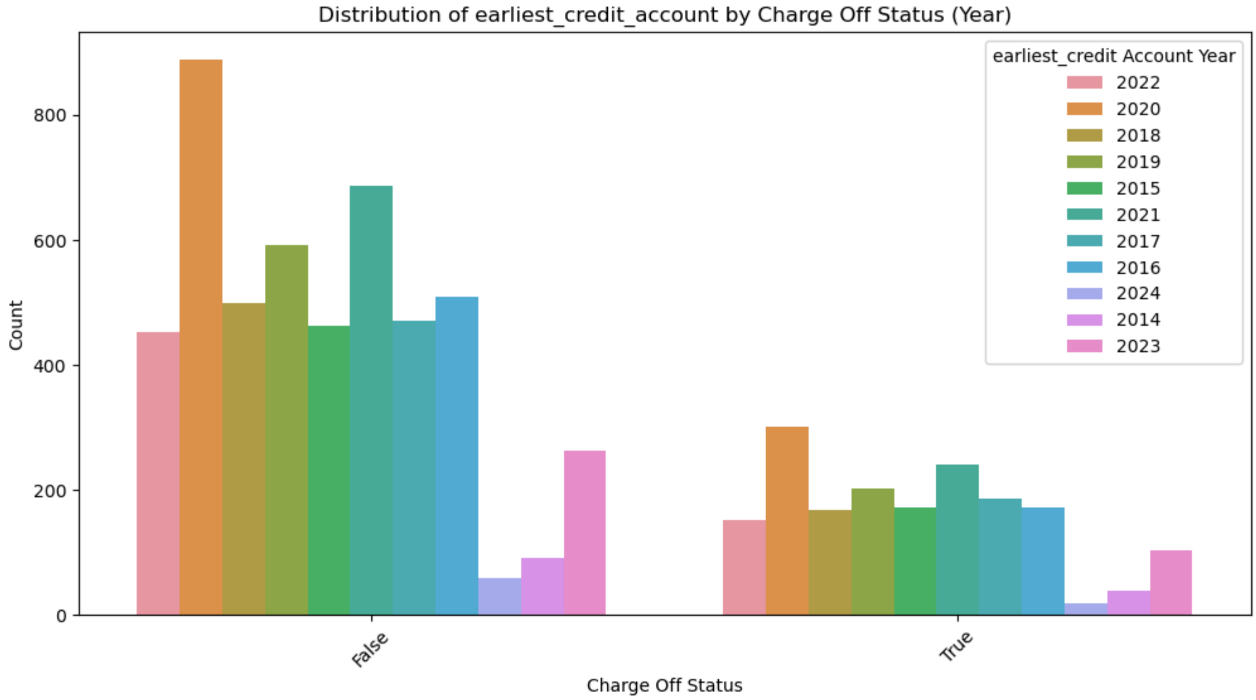


In [200]:
```
plt.figure(figsize=(20, 6))
sns.countplot(x='charge_off_status', hue=df['earliest_credit_account'].dt.to_period('M'), data=df)
plt.title('Distribution of earliest_credit_account  by Charge Off Status(month and year)')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.legend().remove()
plt.show()
```

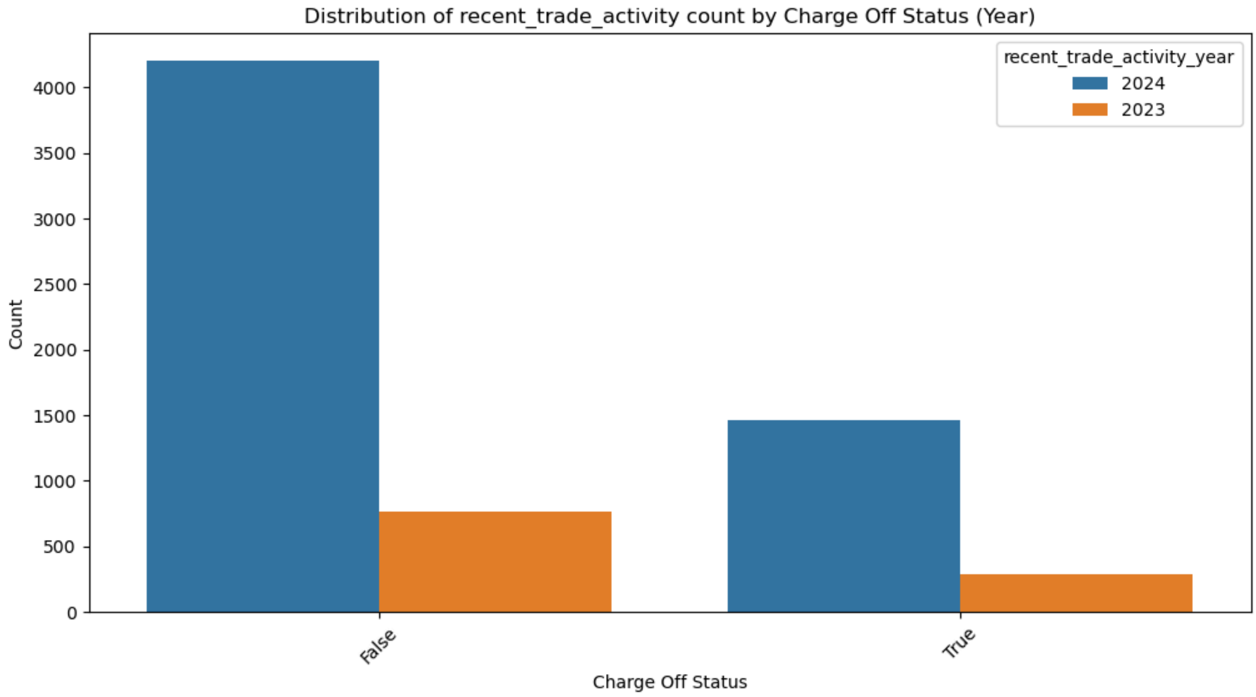


In [201]:

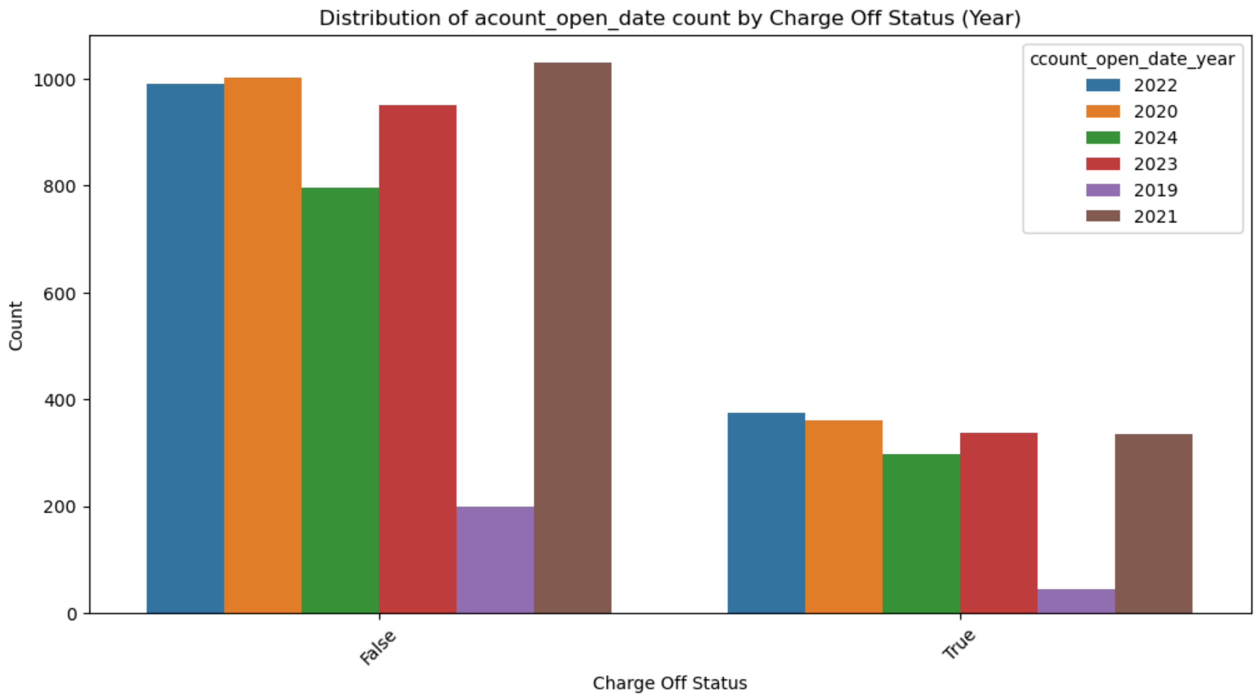
```
df_new=df.copy()
# Ensure 'earliest_credit_account' is in datetime format
df_new['earliest_credit_account'] = pd.to_datetime(df['earliest_credit_account'], errors='coerce')

# Extract month and year and create a new column for plotting
df_new['credit_account_year'] = df_new['earliest_credit_account'].dt.to_period('Y')


plt.figure(figsize=(12, 6))
sns.countplot(x='charge_off_status', hue='credit_account_year', data=df_new)
plt.title('Distribution of earliest_credit_account by Charge Off Status (Year)')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.legend(title='earliest_credit Account Year')

plt.show()
```



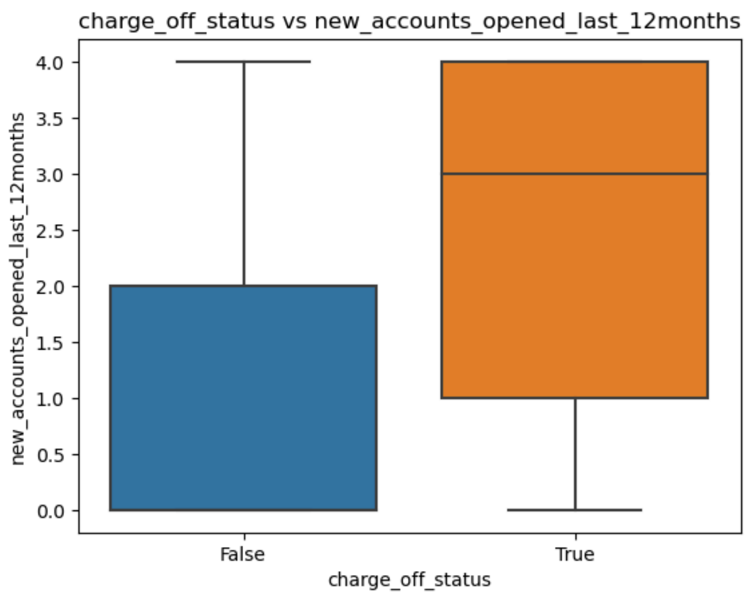
In [202]:
```
df_new=df.copy()
# Ensure 'earliest_credit_account' is in datetime format
df_new['recent_trade_activity'] = pd.to_datetime(df['recent_trade_activity'], errors='coerce')

# Extract month and year and create a new column for plotting
df_new['recent_trade_activity_year'] = df_new['recent_trade_activity'].dt.to_period('Y')


plt.figure(figsize=(12, 6))
sns.countplot(x='charge_off_status', hue='recent_trade_activity_year', data=df_new)
plt.title('Distribution of recent_trade_activity count by Charge Off Status (Year)')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.legend(title='recent_trade_activity_year')

plt.show()
```



In [203]:
```
df_new=df.copy()
# Ensure 'earliest_credit_account' is in datetime format
df_new['account_open_date'] = pd.to_datetime(df['account_open_date'], errors='coerce')

# Extract month and year and create a new column for plotting
df_new['account_open_date_year'] = df_new['account_open_date'].dt.to_period('Y')


plt.figure(figsize=(12, 6))
sns.countplot(x='charge_off_status', hue='account_open_date_year', data=df_new)
plt.title('Distribution of acount_open_date count by Charge Off Status (Year)')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.legend(title='ccount_open_date_year')

plt.show()
```

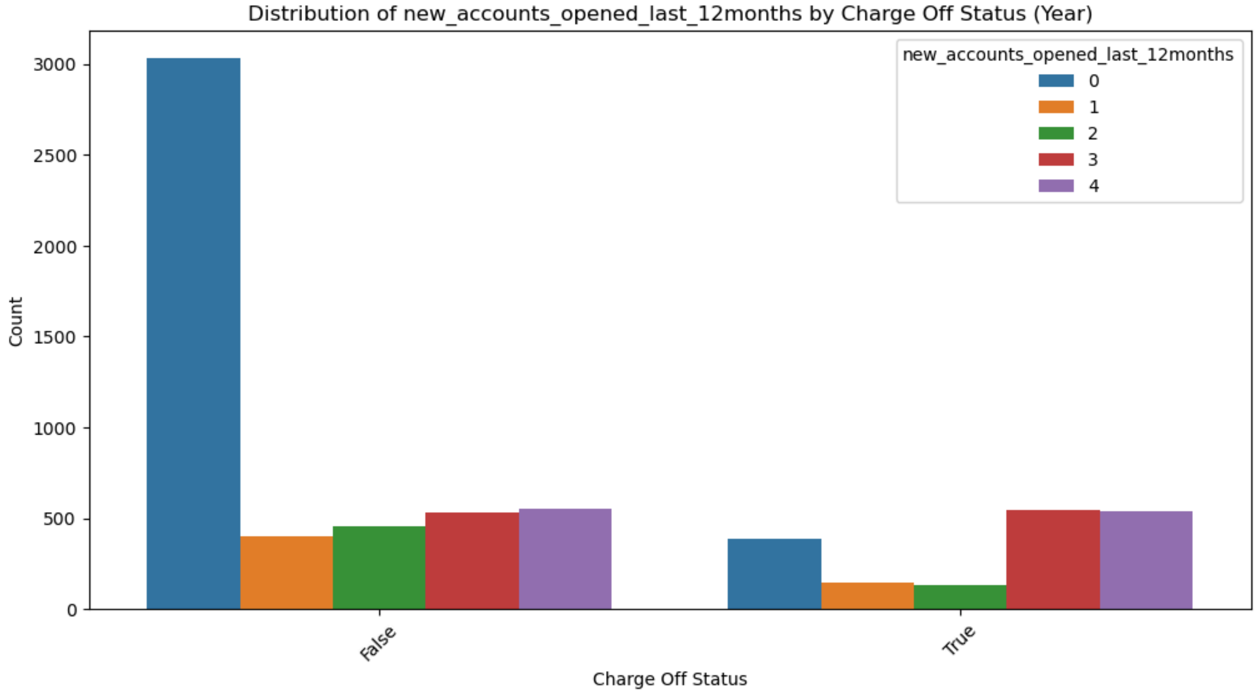


In [204]:
```
sns.boxplot(x=df['charge_off_status'], y=df['new_accounts_opened_last_12months'])
plt.title('charge_off_status vs new_accounts_opened_last_12months')
plt.xlabel('charge_off_status')

plt.show()
```



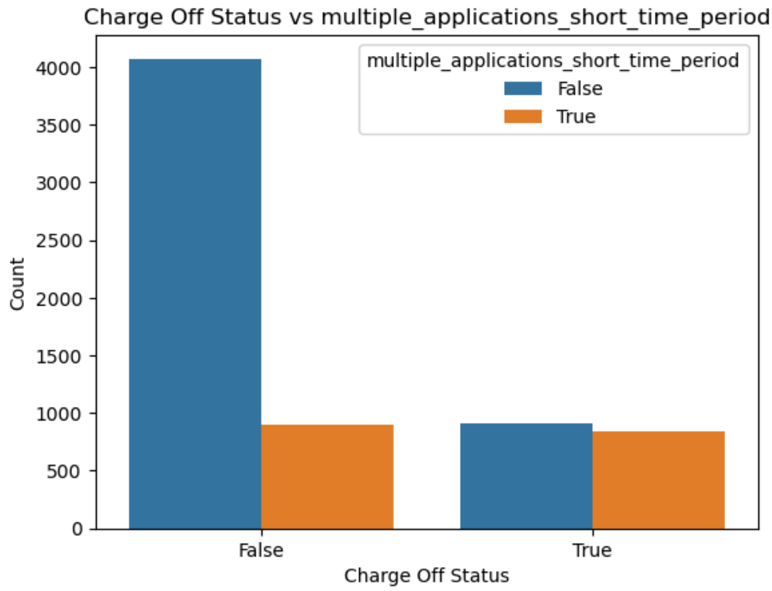
In [205]:

```
plt.figure(figsize=(12, 6))
sns.countplot(x='charge_off_status', hue='new_accounts_opened_last_12months', data=df)
plt.title('Distribution of new_accounts_opened_last_12months by Charge Off Status (Year)')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.xticks(rotation=45)


plt.show()
```

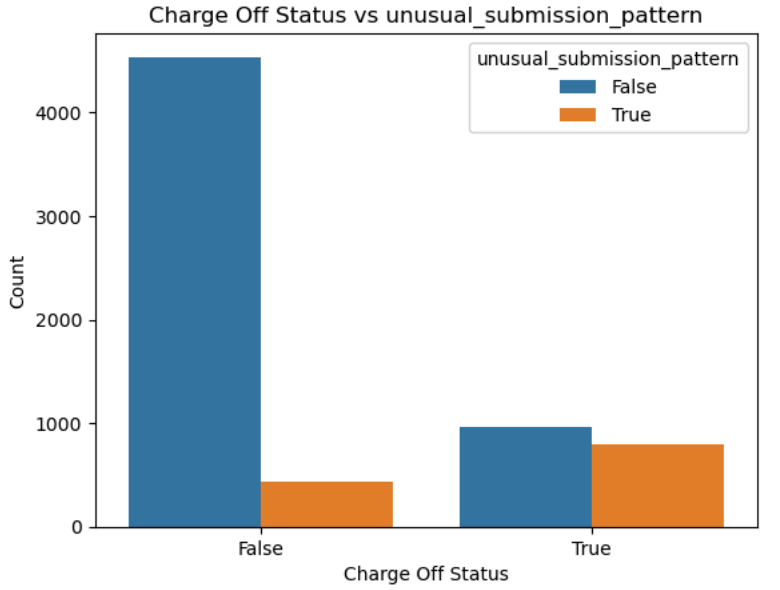


In [206]:

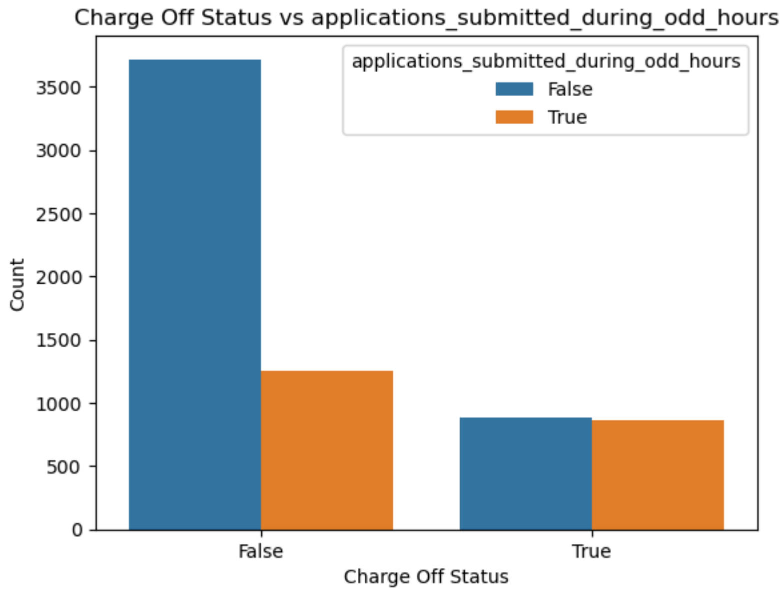
```
sns.countplot(x='charge_off_status', hue='multiple_applications_short_time_period', data=df)
plt.title('Charge Off Status vs multiple_applications_short_time_period')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.show()
```



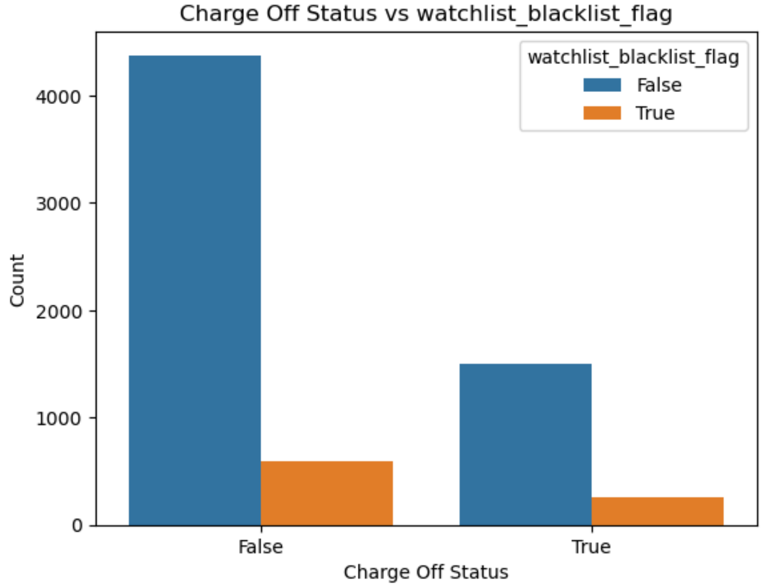
In [207]:
```
sns.countplot(x='charge_off_status', hue='unusual_submission_pattern', data=df)
plt.title('Charge Off Status vs unusual_submission_pattern')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.show()
```



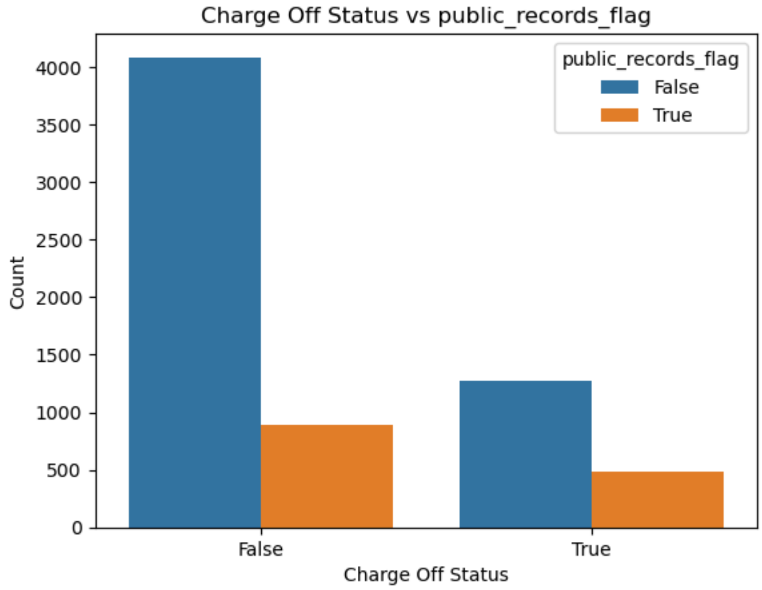
In [208]:
```
sns.countplot(x='charge_off_status', hue='applications_submitted_during_odd_hours', data=df)
plt.title('Charge Off Status vs applications_submitted_during_odd_hours')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.show()
```



In [209]:
```
sns.countplot(x='charge_off_status', hue='watchlist_blacklist_flag', data=df)
plt.title('Charge Off Status vs watchlist_blacklist_flag')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.show()
```



In [210]:
```
sns.countplot(x='charge_off_status', hue='public_records_flag', data=df)
plt.title('Charge Off Status vs public_records_flag')
plt.xlabel('Charge Off Status')
plt.ylabel('Count')
plt.show()
```



In [211]:
```
df
```

Out[211]:

	account_open_date	age	location	occupation	income_level	fico_score	delinquency_status	charge_off_status	number_of_credit_applications	debt_
0	2022-09-11	69	New York	Engineer	2050	483.0	0	False	1	
1	2020-07-12	46	Miami	Engineer	71936	566.0	0	False	1	
2	2024-07-27	60	Houston	Lawyer	8574	787.0	0	False	1	
3	2022-03-06	25	Chicago	Teacher	59349	688.0	0	False	1	
4	2023-10-26	38	Los Angeles	Banker	6925	740.0	0	False	1	
...	...	...	...	...	...	...	...	...	...	
6715	2019-12-27	74	Chicago	Doctor	51012	580.0	93	True	3	
6716	2021-04-10	33	Boston	Consultant	14566	589.0	92	True	2	
6717	2022-03-18	66	Atlanta	Banker	8930	679.0	0	True	1	
6718	2024-06-20	68	Houston	Other	4501	321.0	58	True	1	
6719	2020-06-13	52	Seattle	Banker	7861	359.0	103	True	5	

6720 rows × 28 columns

In [212]:
```
df['charge_off_status'] =df.pop('charge_off_status')
```

In [213]:
```
df.dtypes
```

Out[213]:
```
account_open_date                      datetime64[ns]
age                                             int64
location                                       object
occupation                                     object
income_level                                    int64
fico_score                                    float64
delinquency_status                              int64
number_of_credit_applications                   int64
debt_to_income_ratio                          float64
payment_methods_high_risk                        bool
max_balance                                   float64
avg_balance_last_12months                     float64
number_of_delinquent_accounts                 float64
number_of_defaulted_accounts                    int64
earliest_credit_account                datetime64[ns]
recent_trade_activity                  datetime64[ns]
new_accounts_opened_last_12months               int64
multiple_applications_short_time_period          bool
unusual_submission_pattern                       bool
applications_submitted_during_odd_hours          bool
watchlist_blacklist_flag                         bool
public_records_flag                              bool
date_contradiction_flag                          bool
delinquency_status_log                        float64
number_of_credit_applications_log             float64
max_balance_log                               float64
avg_balance_last_12months_capped              float64
charge_off_status                                bool
dtype: object
```

In [214]:
```
df_clean = df.drop(['delinquency_status', 'number_of_credit_applications', 'max_balance', 'avg_balance_last_12months'], axis
```

In [215]:
```
df_clean.dtypes
```

Out[215]:
```
account_open_date                       datetime64[ns]
age                                              int64
location                                        object
occupation                                      object
income_level                                     int64
fico_score                                     float64
debt_to_income_ratio                           float64
payment_methods_high_risk                         bool
number_of_delinquent_accounts                  float64
number_of_defaulted_accounts                     int64
earliest_credit_account                 datetime64[ns]
recent_trade_activity                   datetime64[ns]
new_accounts_opened_last_12months                int64
multiple_applications_short_time_period           bool
unusual_submission_pattern                        bool
applications_submitted_during_odd_hours           bool
watchlist_blacklist_flag                          bool
public_records_flag                               bool
date_contradiction_flag                           bool
delinquency_status_log                         float64
number_of_credit_applications_log              float64
max_balance_log                                float64
avg_balance_last_12months_capped               float64
charge_off_status                                 bool
dtype: object
```

In [216]:
```
df_clean
```

Out[216]:

	account_open_date	age	location	occupation	income_level	fico_score	debt_to_income_ratio	payment_methods_high_risk	number_of_delinquent_ac
0	2022-09-11	69	New York	Engineer	2050	483.0	1.017489	False	
1	2020-07-12	46	Miami	Engineer	71936	566.0	1.508626	False	
2	2024-07-27	60	Houston	Lawyer	8574	787.0	1.182380	False	
3	2022-03-06	25	Chicago	Teacher	59349	688.0	1.121196	True	
4	2023-10-26	38	Los Angeles	Banker	6925	740.0	1.659631	False	
...	...	...	...	...	...	...	...	...	
6715	2019-12-27	74	Chicago	Doctor	51012	580.0	2.648878	True	
6716	2021-04-10	33	Boston	Consultant	14566	589.0	1.727134	True	
6717	2022-03-18	66	Atlanta	Banker	8930	679.0	1.430593	False	
6718	2024-06-20	68	Houston	Other	4501	321.0	1.750392	True	
6719	2020-06-13	52	Seattle	Banker	7861	359.0	1.751935	False	

6720 rows × 24 columns

In [217]:
```
df_clean .to_csv('df_cleaned.csv', index=False)
```

In [ ]:

In [ ]: